

TÀI LIỆU THỰC HÀNH

MÔN

CHUYÊN ĐỀ

ASP.NET

CHO SINH VIÊN CNTT

Xây dựng ứng dụng Full-stack chuyên nghiệp
với **ASP.NET Core & ReactJS**



.NET 8.0
Hiệu năng vượt trội



Bảo mật
và ổn định



Kiến trúc
3 lớp chuẩn



ReactJS
Hiện đại



Sẵn sàng
triển khai



THS. NGUYỄN CAO THÁI

2024

PHẦN 1: NỀN TẢNG & CƠ SỞ DỮ LIỆU (CHƯƠNG 1 & 2)

Buổi 1: Tổng quan .NET Core & C# Nâng cao

- **Mục tiêu:** Cài đặt môi trường và hiểu cấu trúc Solution 3 lớp.
- **Nội dung:** Giới thiệu .NET Core, Dependency Injection cơ bản, và cấu trúc thư mục Web API.
- **Thực hành:** Khởi tạo Base Project; tạo các Class Entity đầu tiên (Category, Post).
- **Giải thích:** Giúp làm quen với Visual Studio và cách tổ chức code chuyên nghiệp ngay từ đầu.

Buổi 2: EF Core & Migration (Trọng tâm Chương 2)

- **Mục tiêu:** Thiết lập kết nối CSDL và tạo bảng tự động.
- **Nội dung:** Cấu hình DbContext, Connection String trong appsettings.json, và các lệnh Migration.
- **Thực hành:** Chạy Add-Migration và Update-Database để sinh ra CSDL CMS_DB.
- **Giải thích:** Đây là bước "xương sống" để quản lý dữ liệu mà không cần viết SQL thủ công.

Buổi 3: Truy vấn LINQ & Thao tác dữ liệu chuyên sâu

- **Mục tiêu:** Thành thạo các hàm xử lý dữ liệu của EF Core.
 - **Nội dung:** Truy vấn LINQ (Select, Where, Include), cách thêm/sửa/xóa dữ liệu.
 - **Thực hành:** Viết các hàm Logic xử lý dữ liệu cho Danh mục và Bài viết.
 - **Giải thích:** Giúp hiểu cách lấy dữ liệu từ SQL lên đối tượng C# để xử lý.
-

PHẦN 2: BACKEND API & ADMIN PANEL (CHƯƠNG 3 & 4)

Buổi 4: ASP.NET Core MVC cho trang Quản trị

- **Mục tiêu:** Xây dựng giao diện nhập liệu cho Admin.
- **Nội dung:** Controller, View, Layout, và HTML Helpers.
- **Thực hành:** Tạo các trang liệt kê và form thêm mới Danh mục/Bài viết bằng Razor View.
- **Giải thích:** cần nắm vững MVC để làm các trang quản trị nhanh chóng.

Buổi 5: Validation & Identity (Bảo mật)

- **Mục tiêu:** Kiểm soát dữ liệu và phân quyền truy cập.
- **Nội dung:** Data Annotations (kiểm tra tính hợp lệ) và ASP.NET Core Identity.
- **Thực hành:** Thiết lập trang Đăng nhập cho Admin và ràng buộc dữ liệu cho form bài viết.
- **Giải thích:** Đảm bảo chỉ người có quyền mới được sửa nội dung website.

Buổi 6: WebAPI RESTful Service (Trọng tâm Chương 4)

- **Mục tiêu:** Cung cấp "cửa ngõ" dữ liệu cho ReactJS.
 - **Nội dung:** Định nghĩa Route, HTTP Methods (GET, POST, PUT, DELETE), và định dạng JSON.
 - **Thực hành:** Xây dựng các API lấy danh sách bài viết theo danh mục và chi tiết bài viết.
 - **Giải thích:** API là cầu nối để Backend và Frontend "nói chuyện" với nhau.
-

PHẦN 3: FRONTEND REACTJS & KẾT NỐI (CHƯƠNG 4)

Buổi 7: Nhập môn ReactJS cho .NET

- **Mục tiêu:** Hiểu cách xây dựng giao diện bằng Component.
- **Nội dung:** Cài đặt Node.js, tạo Project React, JSX, Props, và State.
- **Thực hành:** Tạo giao diện "Card" bài viết đơn giản.
- **Giải thích:** Chuyển đổi từ duy từ Render phía Server sang Render phía Client.

Buổi 8: Gọi API từ ReactJS & useEffect

- **Mục tiêu:** Hiển thị dữ liệu thực tế từ Database lên ReactJS.
- **Nội dung:** Thư viện Axios, vòng đời Component (useEffect) để gọi API.
- **Thực hành:** Gọi API bài viết từ Backend và hiển thị lên trang chủ React.
- **Giải thích:** Đây là bước thực hiện mục tiêu "Sử dụng giao diện ReactJS gọi WebAPI" trong đề cương.

Buổi 9: Routing & Trang chi tiết bài viết

- **Mục tiêu:** Điều hướng người dùng trong ứng dụng React.
- **Nội dung:** React Router Dom, truyền ID bài viết qua URL.
- **Thực hành:** Làm chức năng nhấn vào tiêu đề bài viết để mở trang xem chi tiết nội dung.
- **Giải thích:** Giúp website hoạt động mượt mà không cần tải lại toàn bộ trang (SPA).

PHẦN 4: NÂNG CAO & HOÀN THIỆN ĐỒ ÁN

Buổi 10: gRPC Service & Kiểm tra tiến độ (Chương 2 nâng cao)

- **Mục tiêu:** Giới thiệu giao tiếp tốc độ cao và chốt điểm quá trình.
- **Nội dung:** Tổng quan gRPC, xây dựng Service đơn giản bằng gRPC.
- **Thực hành:** Kiểm tra (test) tiến độ đồ án của từng nhóm.
- **Giải thích:** gRPC là nội dung chuyên sâu giúp có thêm điểm cộng về công nghệ mới.

Buổi 11: Hoàn thiện Website thực tế & Debug

- **Mục tiêu:** Gỡ lỗi và tối ưu hóa trải nghiệm người dùng.
- **Nội dung:** Kỹ năng gỡ lỗi (Debug) trong Visual Studio và Chrome DevTools.
- **Thực hành:** Hướng dẫn sửa các lỗi giao diện, lỗi API và tối ưu SEO cơ bản cho bài viết.
- **Giải thích:** Rèn luyện tính cẩn thận và thẩm mỹ trước khi báo cáo tốt nghiệp.

Buổi 12: Tổng kết & Báo cáo đồ án kết thúc môn

- **Mục tiêu:** Đánh giá năng lực đạt được.
- **Nội dung:** Thuyết trình, Demo ứng dụng CMS Full-stack.
- **Đánh giá:** Chấm điểm dựa trên khả năng vận dụng kiến thức ASP.NET Core, ReactJS và tính hoàn thiện của đồ án.

===

HƯỚNG DẪN CÀI ĐẶT MÔI TRƯỜNG LẬP TRÌNH ASP.NET CORE

Mục tiêu: Thiết lập đầy đủ "xưởng sản xuất" phần mềm trên máy tính cá nhân để thực hiện dự án website bán hàng tích hợp tin tức.

1. CÀI ĐẶT VISUAL STUDIO 2022 (CÔNG CỤ LẬP TRÌNH CHÍNH)

Visual Studio là IDE (môi trường phát triển tích hợp) mạnh mẽ nhất để viết code C# và ASP.NET.

- **Bước 1:** Truy cập visualstudio.microsoft.com.
- **Bước 2:** Chọn phiên bản **Community 2022** (Bản miễn phí dành cho sinh viên).
- **Bước 3:** Khi chạy bộ cài (Installer), tại tab **Workloads**, sinh viên **BẮT BUỘC** phải tích chọn:
 - **ASP.NET and web development** (Để lập trình Website).
 - **.NET desktop development** (Để hỗ trợ các thư viện hệ thống).
- **Bước 4:** Nhấn **Install** và đợi máy tải về (khoảng 2-5GB tùy tốc độ mạng).

Lưu ý: Nếu máy tính của sinh viên có cấu hình yếu (RAM dưới 8GB), có thể cân nhắc dùng **Visual Studio Code**, nhưng trong khuôn khổ môn học này, dùng Visual Studio 2022 sẽ hỗ trợ kéo-thả và quản lý Database tốt nhất.

2. CÀI ĐẶT SQL SERVER & SSMS (HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU)

Để lưu trữ sản phẩm thời trang, đơn hàng và khách hàng, chúng ta cần "nhà kho" SQL Server.

A. SQL Server Express

- **Tải về:** Chọn bản **Express** (bản rút gọn, miễn phí).
- **Cài đặt:** Chọn kiểu cài đặt **Basic**.

B. SQL Server Management Studio (SSMS)

Đây là phần mềm có giao diện để chúng ta nhìn thấy các bảng dữ liệu (như bài thực hành vẽ Diagram vừa rồi).

- **Tải về:** [Tải SSMS tại đây](#).
- **Kiểm tra:** Sau khi cài xong, mở SSMS, nếu kết nối thành công vào `(localdb)\MSSQLLocalDB` hoặc `.\SQLEXPRESS` là đạt yêu cầu.

3. CÀI ĐẶT .NET 6.0 HOẶC .NET 8.0 SDK

SDK (Software Development Kit) là bộ thư viện nền tảng để chạy code ASP.NET.

- **Kiểm tra:** Mở cửa sổ Command Prompt (gõ `cmd` ở nút Start), nhập lệnh: `dotnet --version`.
- **Nếu chưa có:** Tải bản **LTS (Long Term Support)** tại dotnet.microsoft.com. Hiện tại .NET 8.0 là bản mới và ổn định nhất.

4. CÁC TIỆN ÍCH HỖ TRỢ (NÊN CÓ)

Để việc học trở nên dễ dàng hơn, giảng viên khuyến khích sinh viên cài thêm:

1. **Postman:** Công cụ để chạy thử (test) các API như `OrderController` mà không cần chờ giao diện Frontend.
2. **JSON Viewer (Extension trên Chrome):** Giúp xem dữ liệu sản phẩm trả về từ API đẹp và dễ đọc hơn.
3. **Git for Windows:** Để lưu trữ code lên GitHub, phục vụ cho việc nộp bài báo cáo cuối môn.

5. KIỂM TRA SAU CÀI ĐẶT (CHECKPOINT)

Để xác nhận máy tính đã sẵn sàng, sinh viên thực hiện:

1. Mở Visual Studio 2022.
2. Chọn **Create a new project**.
3. Tìm mẫu **ASP.NET Core Web API**.
4. Nhấn tạo và ấn **F5**. Nếu trình duyệt hiện lên trang **Swagger** (trang danh sách các hàm API) nghĩa là việc cài đặt đã hoàn tất 100%.

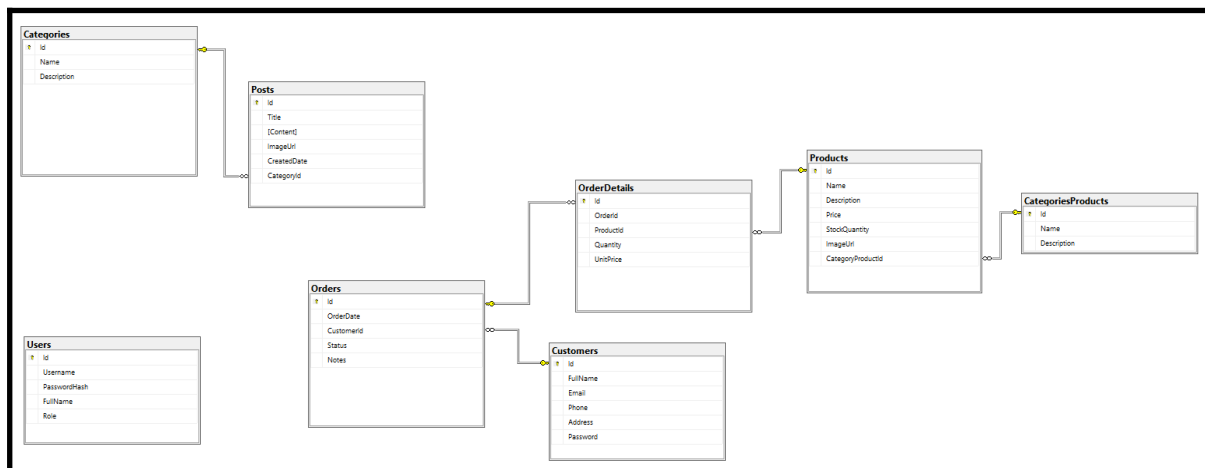
BUỔI 1: KHỞI TẠO CẤU TRÚC ĐỒ ÁN TỐT NGHIỆP CMS FULL-STACK



Mục tiêu bài học

Sau khi kết thúc buổi này, có thể:

- Tự thiết lập cấu trúc Solution 3 lớp chuyên nghiệp (Data - Backend - Frontend).
- Định nghĩa toàn bộ các thực thể (Entities) cần thiết cho một hệ thống quản trị nội dung.
- Hiểu mối quan hệ giữa các bảng trong Cơ sở dữ liệu (Database Design).



Bước 1: Tạo Thùng chứa dự án (Blank Solution)

1. Mở **Visual Studio 2022**.
2. Chọn **Create a new project**.
3. Tìm từ khóa "**Blank Solution**" -> Chọn và bấm **Next**.
4. *Solution name:* **ThaiCMS_Solution**. Bấm **Create**.



Bước 2: Tạo Project 1 - **CMS.Data** (Lớp Dữ liệu)

Đây là nơi chứa "linh hồn" của ứng dụng: các bảng dữ liệu.

1. Chuột phải vào Solution -> **Add** -> **New Project**.
2. Chọn mẫu **Class Library (.NET Core)**. Bấm **Next**.
3. *Project name:* **CMS.Data**. Bấm **Create**.
4. Xóa file **Class1.cs**. Tạo thư mục tên là **Entities**.
5. **Tạo 8 Class quan trọng sau bên trong thư mục Entities:**

2.1. Thực thể Danh mục (**Category.cs**)

C#

```
namespace CMS.Data.Entities {  
    public class Category {  
        public int Id { get; set; }  
        public string Name { get; set; } // Tên danh mục (vd: Tin Giáo Dục)
```

```
        public string Description { get; set; }

        // Quan hệ: Một danh mục có nhiều bài viết
        public virtual ICollection<Post> Posts { get; set; }
    }
}
```

2.2. Thực thể Bài viết (**Post.cs**)

C#

```
namespace CMS.Data.Entities {
    public class Post {
        public int Id { get; set; }
        public string Title { get; set; } // Tiêu đề bài viết
        public string Content { get; set; } // Nội dung chi tiết
        public string ImageUrl { get; set; } // Hình ảnh đại diện
        public DateTime CreatedDate { get; set; } = DateTime.Now;

        // Khóa ngoại liên kết tới Category
        public int CategoryId { get; set; }
        public virtual Category Category { get; set; }
    }
}
```

2.3. Thực thể Người dùng (**User.cs**)

Dùng cho chức năng đăng nhập quản trị (Membership).

C#

```
namespace CMS.Data.Entities {
    public class User {
        public int Id { get; set; }
        public string Username { get; set; }
        public string PasswordHash { get; set; }
        public string FullName { get; set; }
        public string Role { get; set; } // Quản trị viên hoặc Biên tập viên
    }
}
```

2.4. Thực thể Danh mục sản phẩm (**CategoryProduct.cs**)

C#

```
using System.ComponentModel.DataAnnotations;
namespace CMS.Data.Entities
{
    public class CategoryProduct
    {
        [Key]
        public int Id { get; set; }
```

```
[Required(ErrorMessage = "Tên danh mục không được để trống")]
[StringLength(100)]
public string Name { get; set; }

public string? Description { get; set; }

// Quan hệ: Một danh mục có nhiều sản phẩm
public virtual ICollection<Product>? Products { get; set; }
}
}
```

2.5. Thực thể Sản phẩm (**Product.cs**)

C#

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace CMS.Data.Entities
{
    public class Product
    {
        [Key]
        public int Id { get; set; }

        [Required(ErrorMessage = "Tên sản phẩm không được để trống")]
        public string Name { get; set; }

        public string? Description { get; set; }

        [Range(0, double.MaxValue)]
        [Column(TypeName = "decimal(18,2)")]
        public decimal Price { get; set; }

        public int StockQuantity { get; set; }

        public string? ImageUrl { get; set; }

        // Khóa ngoại nối tới CategoryProduct
        public int CategoryProductId { get; set; }

        [ForeignKey("CategoryProductId")]
        public virtual CategoryProduct? CategoryProduct { get; set; }
    }
}
```

2.6. Thực thể Khách hàng (**Customer.cs**)

C#

```
using System.ComponentModel.DataAnnotations;

namespace CMS.Data.Entities
{
    // Khách hàng
    public class Customer
    {
        [Key]
        public int Id { get; set; }

        [Required]
        public string FullName { get; set; }

        [Required]
        [EmailAddress]
        public string Email { get; set; }

        public string? Phone { get; set; }

        public string? Address { get; set; }

        [Required]
        public string Password { get; set; } // Lưu mật khẩu thô theo yêu cầu tối giản

        public virtual ICollection<Order>? Orders { get; set; }
    }
}
```

2.7. Thực thể Đơn hàng (Order.cs)

C#

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace CMS.Data.Entities
{
    public class Order
    {
        [Key]
        public int Id { get; set; }

        public DateTime OrderDate { get; set; } = DateTime.Now;

        public int CustomerId { get; set; }

        public int Status { get; set; } // 0: Chờ duyệt, 1: Đang giao, 2: Đã xong
    }
}
```

```
public string? Notes { get; set; }

[ForeignKey("CustomerId")]
public virtual Customer? Customer { get; set; }

public virtual ICollection<OrderDetail>? OrderDetails { get; set; }
}
}
```

2.8. Thực thể Chi tiết đơn hàng (**OrderDetail.cs**)

C#

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace CMS.Data.Entities
{
    public class OrderDetail
    {
        [Key]
        public int Id { get; set; }

        public int OrderId { get; set; }

        public int ProductId { get; set; }

        public int Quantity { get; set; }

        [Column(TypeName = "decimal(18,2)")]
        public decimal UnitPrice { get; set; } // Giá tại thời điểm mua

        [ForeignKey("OrderId")]
        public virtual Order? Order { get; set; }

        [ForeignKey("ProductId")]
        public virtual Product? Product { get; set; }
    }
}
```

Bước 3: Tạo Project 2 - CMS.Backend (Lớp Xử lý API)

Project này dùng công nghệ ASP.NET Core MVC để làm trang Admin và WebAPI cho ReactJS.

1. Chuột phải vào Solution -> **Add** -> **New Project**.
2. Chọn mẫu **ASP.NET Core Web App (Model-View-Controller)**. Bấm **Next**.
3. *Project name*: **CMS.Backend**. Bấm **Create**.

4. **Kết nối:** Chuột phải vào **CMS.Backend** -> **Add** -> **Project Reference** -> Tích chọn **CMS.Data**.

Kiểm chứng Project hoạt động chưa? Thiết lập dự án khởi động (Startup Project)

Trong một Solution có nhiều Project, Visual Studio cần biết bạn muốn "chạy" cái nào đầu tiên.

1. Tại cửa sổ **Solution Explorer**, tìm project **CMS.Backend**.
2. **Chuột phải** vào tên **CMS.Backend** -> Chọn dòng **Set as Startup Project**.
3. **Dấu hiệu thành công:** Tên project **CMS.Backend** sẽ được **in đậm** lên.

Nhấn phím **F5** trên bàn phím (hoặc nhấn vào nút tam giác xanh có chữ **CMS.Backend** trên thanh công cụ).

Xác nhận bảo mật: Nếu máy hiện bảng hỏi về **SSL Certificate** (HTTPS), hãy nhấn **Yes** liên tục để trình duyệt tin cậy địa chỉ **localhost**.

Kết quả:

Phải hiện ra trang chào mừng của ASP.NET Core (Welcome) tại địa chỉ **https://localhost:xxxx/**



Bước 4: Tạo Project 3 - **CMS.Frontend** (Lớp Giao diện)

Project này dùng ReactJS để hiển thị tin tức cho người xem.

Bước 4.1- Cài đặt môi trường chạy ReactJS:

1. **Tải Node.js:** Truy cập trang chủ nodejs.org.
2. **Chọn phiên bản:** Tải bản **LTS** (Long Term Support - Bản ổn định nhất).
3. **Cài đặt:** Mở file vừa tải về, nhấn **Next** liên tục cho đến khi hoàn tất. (Lưu ý: Tích chọn vào ô "Automatically install the necessary tools" nếu có).
4. **Khởi động lại:** Sau khi cài xong, **BẮT BUỘC** phải tắt hẳn Visual Studio.

Kiểm chứng [Node.js](https://nodejs.org) đã hoạt động bằng cách gõ lệnh

```
node -v
```

hay

```
npm -v
```

Bước 4.2- Chạy chương trình Windows Explorer, tìm và mở folder Solution, nhấp chuột vào thanh địa chỉ của folder, gõ lệnh **cmd** (enter), mục đích để mở cửa sổ lệnh.

Gõ lệnh sau để tạo project React (nhẹ và ổn định hơn):

```
npx create-react-app cms.frontend
```

Bước 4.3- Sau khi lệnh chạy xong, quay lại Visual Studio -> Chuột phải Solution -> **Add** -> Existing Web Site...

Tìm đến đúng thư mục cms.frontend.

Nhấn Open.

VS Studio 2022 sẽ hỏi xác nhận cập nhật .NET framework

Lựa chọn: "Update the target to .NET Framework 4.8 (Recommended)"

Sau đó nhấn **Continue**.

Giải thích

Bản chất: Dự án React của chúng ta chạy bằng Node.js, nó hoàn toàn không dùng đến .NET Framework 4.0 hay 4.8.

Mục đích: Việc chọn "Update" chỉ là để Visual Studio tạo ra một file cấu hình tạm thời giúp nó hiển thị được cây thư mục (node_modules, src, public) trong cửa sổ Solution Explorer. Điều này giúp có thể mở file .js hoặc .css để sửa code trực tiếp trong Visual Studio cho tiện.

Kết quả: Thư mục sẽ hiện lên trong Solution Explorer với đầy đủ các file src, public, package.json.

Cách chạy thử Project 3 sau khi đã Add thành công:

Vì đây là "Existing Web Site", nút Play (F5) của Visual Studio có thể sẽ không chạy được trang React này. Hãy hướng dẫn cách chạy "chuẩn" của dân Frontend:

1. Chuột phải vào folder **cms.frontend** -> Chọn **Open in Terminal**.
2. Gõ lệnh: **npm run dev** (nếu dùng Vite) hoặc **npm start** (nếu dùng Create React App).
3. Mở trình duyệt theo địa chỉ hiện ra trong Terminal (thường là **localhost:3000** hoặc **5173**).

TM BƯỚC 5: XÁC NHẬN THÀNH QUẢ (DEMO HIỂN THỊ ĐẦU TIÊN)

*Mục tiêu: Giúp thấy được luồng dữ liệu từ Project **Data** đi qua **Backend** và hiện lên trình duyệt.*

5.1. Tạo Controller quản trị

1. Tại Project **CMS.Backend**, chuột phải vào thư mục **Controllers** -> **Add** -> **Controller** -> Chọn **MVC Controller - Empty**.
2. Đặt tên là **CategoryController.cs**.
3. Viết code tạo dữ liệu "giả" để hiển thị:

```
using Microsoft.AspNetCore.Mvc;
using CMS.Data.Entities; // Kết nối tới lớp dữ liệu bạn vừa tạo

public class CategoryController : Controller {
    public IActionResult Index() {
        // Tạo danh sách dữ liệu mẫu trực tiếp trong code
        var list = new List<Category> {
            new Category { Id = 1, Name = "Tin Công Nghệ", Description = "Review Laptop, AI" },
            new Category { Id = 2, Name = "Giáo Dục", Description = "Thông tin tuyển sinh" }
        };
        return View(list); // Gửi danh sách này sang giao diện
    }
}
```

5.2. Tạo Giao diện (View)

1. Trong code trên, chuột phải vào chữ **Index** -> Chọn **Add View**.
2. Chọn **Razor View**, đặt tên là **Index.cshtml**.
3. Dán đoạn code HTML đơn giản sau để hiển thị bảng:

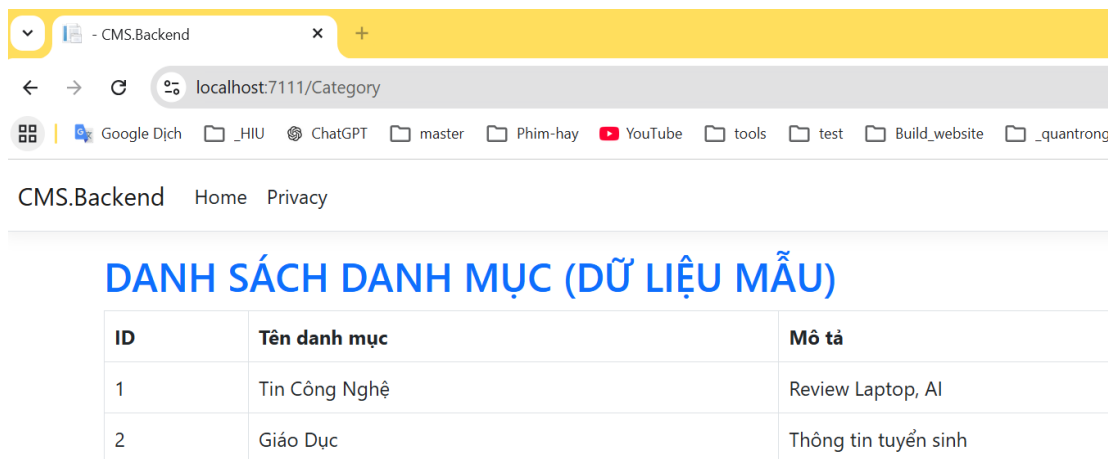
HTML

```
@model IEnumerable<CMS.Data.Entities.Category>

<h2 class="text-primary">DANH SÁCH DANH MỤC (DỮ LIỆU MẪU)</h2>
<table class="table table-bordered">
    <thead>
        <tr>
            <th>ID</th> <th>Tên danh mục</th> <th>Mô tả</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var item in Model) {
            <tr>
                <td>@item.Id</td>
                <td>@item.Name</td>
                <td>@item.Description</td>
            </tr>
        }
    </tbody>
</table>
```

5.3. Chạy dự án

1. Nhấn **F5**.
2. Khi trình duyệt hiện ra, gõ thêm vào thanh địa chỉ: <https://localhost:xxxx/Category>
3. **Kết quả:** sẽ thấy bảng danh mục hiện lên đẹp mắt.



ID	Tên danh mục	Mô tả
1	Tin Công Nghệ	Review Laptop, AI
2	Giáo Dục	Thông tin tuyển sinh

TẠI SAO PHẢI LÀM BƯỚC NÀY?

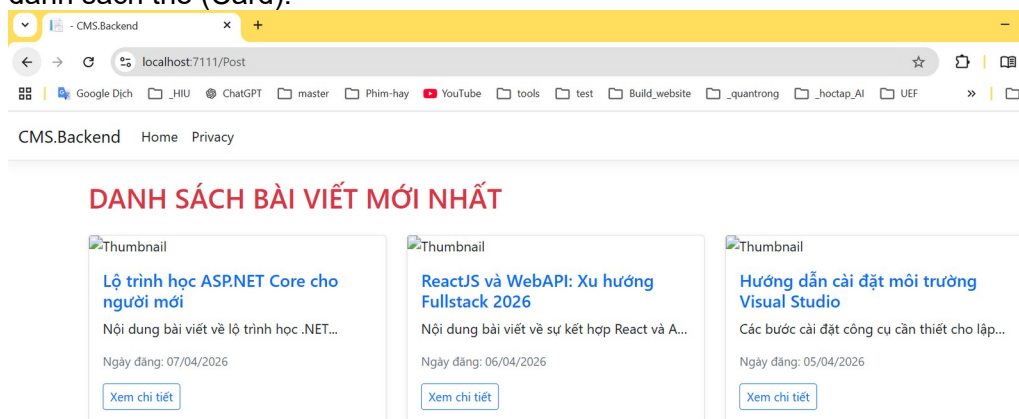
Kiểm tra tính kết nối: Nếu trang web hiện lên đúng tên danh mục, chứng tỏ Project **Backend** đã tham chiếu (Reference) thành công sang Project **Data**.

BÀI TẬP TỰ RÈN LUYỆN (CHALLENGE)

Mục tiêu: Tự tay triển khai luồng dữ liệu cho các thực thể còn lại để làm quen với cấu trúc 3 lớp.

Thử thách 1: Quản lý bài viết (Post)

- Tại Project CMS.Data:** Đảm bảo bạn đã gõ xong class **Post.cs** với các thuộc tính: **Id, Title, Content, ImageUrl**.
- Tại Project CMS.Backend:**
 - Tạo **PostController.cs**.
 - Trong hàm **Index**, hãy tạo một danh sách giả gồm ít nhất 2 bài viết (Ví dụ: "Lộ trình học ASP.NET", "Cài đặt ReactJS").
 - Tạo View **Index.cshtml** để hiển thị danh sách bài viết dưới dạng bảng hoặc danh sách thẻ (Card).



DANH SÁCH BÀI VIẾT MỚI NHẤT		
	Lộ trình học ASP.NET Core cho người mới Nội dung bài viết về lộ trình học .NET... Ngày đăng: 07/04/2026 Xem chi tiết	
	ReactJS và WebAPI: Xu hướng Fullstack 2026 Nội dung bài viết về sự kết hợp React và A... Ngày đăng: 06/04/2026 Xem chi tiết	
	Hướng dẫn cài đặt môi trường Visual Studio Các bước cài đặt công cụ cần thiết cho lập... Ngày đăng: 05/04/2026 Xem chi tiết	

Thử thách 2: Quản lý người dùng (User)

- Tại Project CMS.Data:** Kiểm tra class **User.cs** (Id, Username, FullName, Role).
- Tại Project CMS.Backend:**

- Tạo **UserController.cs**.
- Tạo dữ liệu giả cho 2 tài khoản (Ví dụ: Admin, Editor).
- Tạo View để hiển thị danh sách thành viên quản trị hệ thống.

ID	Tên Đăng Nhập	Họ Và Tên	Quyền Hạn	Thao Tác
1	admin_thai	Nguyễn Cao Thái	Administrator	Sửa Xóa
2	editor_01	Trần Văn Biên Tập	Editor	Sửa Xóa
3	author_minh	Lê Quang Minh	Author	Sửa Xóa

GIẢI THÍCH (TẠI SAO PHẢI LÀM?)

- **Sự lặp lại tạo nên kỹ năng:** Khi bạn làm lại 3 lần cho 3 thực thể khác nhau, tay bạn sẽ quen với việc tạo Controller và View.
- **Hiểu về kiểu dữ liệu:** Bạn sẽ thấy sự khác biệt khi hiển thị một chuỗi ngắn (**Name**) và một chuỗi dài (**Content** của bài viết).
- **Sẵn sàng cho Tuần 2:** Khi sang tuần sau, chúng ta chỉ cần thay thế phần "Dữ liệu giả" này bằng lệnh gọi từ Database qua EF Core là xong 50% đồ án!

Gợi ý:

Code **PostController.cs**

```
using Microsoft.AspNetCore.Mvc;
```

```
using CMS.Data.Entities; // Quan trọng: Phải có dòng này để dùng lớp Post
```

```
namespace CMS.Backend.Controllers
```

```
{
```

```
    public class PostController : Controller
```

```
    {
```

```
        // Hàm Index: Hiển thị danh sách bài viết mẫu
```

```
        public IActionResult Index()
```

```
        {
```

```
            // 1. Tạo dữ liệu giả (Mock Data) cho Bài viết
```

```
            var posts = new List<Post>
```

```
            {
```

```
new Post
{
    Id = 1,
    Title = "Lộ trình học ASP.NET Core cho người mới",
    Content = "Nội dung bài viết về lộ trình học .NET...",
    ImageUrl = "https://via.placeholder.com/150",
    CreatedDate = DateTime.Now
},
new Post
{
    Id = 2,
    Title = "ReactJS và WebAPI: Xu hướng Fullstack 2026",
    Content = "Nội dung bài viết về sự kết hợp React và API...",
    ImageUrl = "https://via.placeholder.com/150",
    CreatedDate = DateTime.Now.AddDays(-1)
},
new Post
{
    Id = 3,
    Title = "Hướng dẫn cài đặt môi trường Visual Studio",
    Content = "Các bước cài đặt công cụ cần thiết cho lập trình viên...",
    ImageUrl = "https://via.placeholder.com/150",
    CreatedDate = DateTime.Now.AddDays(-2)
}
};

// 2. Gửi danh sách dữ liệu sang View
return View(posts);
```

```
}

// Hàm Details: Hiển thị chi tiết một bài viết (Bổ sung khá giỏi)
public IActionResult Details(int id)
{
    // Giả lập tìm bài viết trong Database bằng Id
    // Trong thực tế tuần sau sẽ là: _context.Posts.Find(id);
    var post = new Post
    {
        Id = id,
        Title = "Nội dung chi tiết bài viết số " + id,
        Content = "Đây là nội dung đầy đủ của bài viết mà bạn vừa click vào. Ở đây có thể viết dài hơn để thấy sự khác biệt với trang danh sách.",
        ImageUrl = "https://via.placeholder.com/600x300", // Ảnh to hơn
        CreatedDate = DateTime.Now
    };

    if (post == null) return NotFound();

    return View(post);
}
}
```

Code **CMS.Backend/Views/Post/Index.cshtml**

@model IEnumerable<CMS.Data.Entities.Post>

<div class="container mt-4">

<h2 class="mb-4 text-danger">DANH SÁCH BÀI VIẾT MỚI NHẤT</h2>

```

<div class="row">

    @foreach (var item in Model)
    {
        <div class="col-md-4 mb-3">

            <div class="card shadow-sm">

                <div class="card-body">

                    <h5 class="card-title text-primary">@item.Title</h5>

                    <p class="card-text text-truncate">@item.Content</p>

                    <p class="text-muted small">Ngày đăng:
@item.CreatedDate.ToString("dd/MM/yyyy")</p>

                    <a href="/Post/Details/@item.Id" class="btn btn-outline-primary btn-sm">Xem
chi tiết</a>

                </div>

            </div>

        </div>

    </div>

    }

</div>
</div>

```

Code **UserController.cs**

```

using Microsoft.AspNetCore.Mvc;
using CMS.Data.Entities; // Phải có dòng này để dùng lớp User

namespace CMS.Backend.Controllers
{
    public class UserController : Controller
    {
        // Hàm Index: Hiển thị danh sách thành viên quản trị

        public IActionResult Index()

```

```
{  
    // 1. Tạo danh sách Người dùng giả (Mock Data)  
    var users = new List<User>  
    {  
        new User  
        {  
            Id = 1,  
            Username = "admin_thai",  
            FullName = "Nguyễn Cao Thái",  
            Role = "Administrator"  
        },  
        new User  
        {  
            Id = 2,  
            Username = "editor_01",  
            FullName = "Trần Văn Biên Tập",  
            Role = "Editor"  
        },  
        new User  
        {  
            Id = 3,  
            Username = "author_minh",  
            FullName = "Lê Quang Minh",  
            Role = "Author"  
        }  
    };  
  
    // 2. Trả về View kèm theo danh sách người dùng
```

```
        return View(users);
    }
}
}
```

Code `CMS.Backend/Views/Post/Index.cshtml`

`@model IEnumerable<CMS.Data.Entities.User>`

```
<div class="container mt-4">
```

```
    <div class="d-flex justify-content-between align-items-center mb-3">
```

```
        <h2 class="text-dark">QUẢN LÝ THÀNH VIÊN</h2>
```

```
        <button class="btn btn-success btn-sm">+ Thêm thành viên mới</button>
```

```
    </div>
```

```
<table class="table table-hover shadow-sm border">
```

```
    <thead class="table-light">
```

```
        <tr>
```

```
            <th>ID</th>
```

```
            <th>Tên Đăng Nhập</th>
```

```
            <th>Họ Và Tên</th>
```

```
            <th>Quyền Hạn</th>
```

```
            <th>Thao Tác</th>
```

```
        </tr>
```

```
    </thead>
```

```
    <tbody>
```

```
        @foreach (var user in Model)
```

```
        {
```

```
            <tr>
```

```
                <td>@user.Id</td>
```

```
<td><strong>@user.Username</strong></td>
<td>@user.FullName</td>
<td>
    @if (user.Role == "Administrator")
    {
        <span class="badge bg-danger">@user.Role</span>
    }
    else
    {
        <span class="badge bg-info text-dark">@user.Role</span>
    }
</td>
<td>
    <button class="btn btn-sm btn-outline-warning">Sửa</button>
    <button class="btn btn-sm btn-outline-danger">Xóa</button>
</td>
</tr>
}
</tbody>
</table>
</div>
```

Code CMS.Backend/Views/Post/Details.cshtml

@model CMS.Data.Entities.Post

```
<div class="container mt-5">
    <nav aria-label="breadcrumb">
        <ol class="breadcrumb">
```

```
<li class="breadcrumb-item"><a href="/Post">Danh sách bài viết</a></li>
<li class="breadcrumb-item active">Chi tiết bài viết</li>
</ol>
</nav>

<div class="row justify-content-center">
  <div class="col-md-10">
    <h1 class="display-4 fw-bold text-primary">@Model.Title</h1>
    <p class="text-muted">Đăng ngày: @Model.CreatedDate.ToString("dd/MM/yyyy
HH:mm")</p>
    <hr />

    <div class="text-center mb-4">
      
    </div>

    <div class="post-content fs-5 leading-lg">
      @Model.Content
    </div>

    <div class="mt-5 border-top pt-3">
      <a href="/Post" class="btn btn-secondary">
        <i class="bi bi-arrow-left"></i> Quay lại danh sách
      </a>
    </div>
  </div>
</div>
</div>
</div>
```

=====

BUỔI 2: Kết nối Cơ sở dữ liệu với Entity Framework Core (EF Core).

KẾT NỐI DATABASE VÀ QUẢN TRỊ DỮ LIỆU THẬT XÂY DỰNG CƠ SỞ DỮ LIỆU THẬT VỚI ENTITY FRAMEWORK CORE

Mục tiêu bài học

Sau buổi này, sẽ:

- Cài đặt thành công các thư viện **EF Core** cần thiết.
- Hiểu và thực hiện được kỹ thuật **Code First Migration** (Từ code C# sinh ra bảng SQL).
- Kết nối thành công Project **Backend** với **SQL Server**.
- Thay thế toàn bộ dữ liệu giả ở Buổi 1 bằng dữ liệu thật từ Database.

Nghĩa là:

Sau buổi này, sẽ:

- Biến các lớp C# (**Category**, **Post**, **User**) thành các bảng trong **SQL Server**.
- Biết cách dùng lệnh **Migration** để quản lý lịch sử thay đổi của Database.
- Thay thế toàn bộ "dữ liệu giả" của Buổi 1 bằng "dữ liệu thật" từ máy chủ.

PHẦN 1: CÀI ĐẶT THƯ VIỆN (NUGET PACKAGES)

cần cài đặt 3 "vũ khí" quan trọng vào Project **CMS.Data**:

1. **Microsoft.EntityFrameworkCore.SqlServer**: Để làm việc với SQL Server.
2. **Microsoft.EntityFrameworkCore.Tools**: Để chạy các lệnh tạo bảng (Migration).
3. **Microsoft.EntityFrameworkCore.Design**: Hỗ trợ thiết kế DB.

Cụ thể là: CÀI ĐẶT "CÔNG CỤ XÂY DỰNG" (NUGET PACKAGES)

Để C# có thể nói chuyện được với SQL Server, ta cần cài đặt 3 thư viện hỗ trợ.

1. Tại Visual Studio, vào menu **Tools -> NuGet Package Manager -> Manage NuGet Packages for Solution...**
2. Chọn tab **Browse** và tìm kiếm/cài đặt 3 gói sau vào **CẢ 2 Project** (CMS.Data và CMS.Backend):
 - `Microsoft.EntityFrameworkCore.SqlServer` (Kết nối SQL Server).
 - `Microsoft.EntityFrameworkCore.Tools` (Cung cấp lệnh tạo bảng).
 - `Microsoft.EntityFrameworkCore.Design` (Hỗ trợ thiết kế DB).

Ghi chú: chọn đúng version tương ứng với version .NET

PHẦN 2: TẠO LỚP QUẢN TRỊ KẾT NỐI (APPLICATIONDBCONTEXT)

Đây là "trạm điều khiển" trung tâm để kết nối các Entity (Category, Post, User) với SQL.

Lớp này đóng vai trò như một "người quản kho", giúp chúng ta truy xuất dữ liệu từ các bảng.

1. Trong Project `CMS.Data`, tạo một file mới tên là `ApplicationDbContext.cs`.
2. Chép nội dung sau (Giải thích: `DbSet` tương ứng với một bảng trong SQL):

```
using Microsoft.EntityFrameworkCore;
using CMS.Data.Entities;

namespace CMS.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options) { }

        // Khai báo các bảng dữ liệu
        public DbSet<Category> Categories { get; set; }
        public DbSet<Post> Posts { get; set; }
        public DbSet<User> Users { get; set; }
        public DbSet<CategoryProduct> CategoriesProducts { get; set; }
        public DbSet<Product> Products { get; set; }
        public DbSet<Customer> Customers { get; set; }
        public DbSet<Order> Orders { get; set; }
        public DbSet<OrderDetail> OrderDetails { get; set; }
    }
}
```

```
}
```

PHẦN 3: KỸ THUẬT CODE FIRST MIGRATION

Đây là phần "thần kỳ" nhất mà rất thích: **Biến Class C# thành Bảng SQL Server.**

1. **Cấu hình chuỗi kết nối (Connection String):** Viết vào file `appsettings.json` trong Project **CMS.Backend**.

Cụ thể là: Chúng ta cần chỉ cho trang web biết máy chủ SQL Server đang nằm ở đâu.

- Mở file `appsettings.json` trong Project **CMS.Backend**.
- Thêm đoạn mã sau (Thay `YourServerName` bằng tên máy của bạn, thường là `.` hoặc `(localdb)\mssqllocaldb`):

```
{  
  "ConnectionStrings": {  
    "DefaultConnection":  
    "Server=.;Database=ThaiCMS_DB;Trusted_Connection=True;MultipleActive  
    ResultSets=true;TrustServerCertificate=True"  
  },  
  ...  
}
```

Trường hợp đặc biệt: **Dành cho SQL Express (Bản nhẹ đi kèm Visual Studio)**

Nhiều không cài bản đầy đủ mà dùng bản rút gọn đi kèm khi cài đặt Visual Studio.

- **Đặc điểm:** Tên Server có đuôi `\SQLEXPRESS` hoặc `(localdb)`.

```
"ConnectionStrings": {  
  "DefaultConnection": "Server=(localdb)\\  
mssqllocaldb;Database=ThaiCMS_DB;Trusted_Connection=True;MultipleActi  
veResultSets=true;TrustServerCertificate=True"  
}
```

Giải thích: `(localdb)\mssqllocaldb` là máy chủ ảo siêu nhẹ tích hợp sẵn, rất phù hợp cho các bài tập nhỏ.

2. **Đăng ký Service:** Trong file `Program.cs`, khai báo cho hệ thống biết cách dùng `ApplicationDbContext`.

Đây là bước "cắm điện" để hệ thống bắt đầu nhận diện kết nối Database.

- Mở file `Program.cs` trong Project **CMS.Backend**.
- Thêm 2 dòng này vào đầu file

```
using Microsoft.EntityFrameworkCore;  
  
using CMS.Data;
```

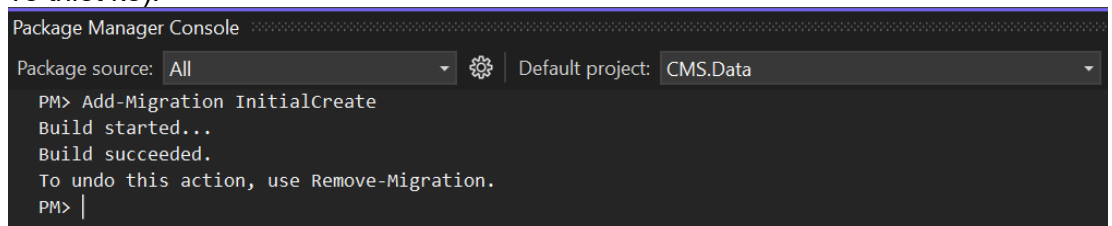
- Thêm đoạn code sau vào trước dòng `var app = builder.Build();`

```
// Đăng ký DbContext vào hệ thống  
  
builder.Services.AddDbContext<ApplicationDbContext>(options =>  
  
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"));
```

3. **Chạy lệnh tại Package Manager Console:** Đây là giây phút quan trọng nhất. Chúng ta sẽ dùng lệnh để SQL Server tự đẻ ra các bảng.
 - o `Add-Migration InitialCreate`: Lập bản vẽ cho Database.
 - o `Update-Database`: Chính thức xây dựng Database trong SQL Server.

Cụ thể là:

1. Vào menu **Tools -> NuGet Package Manager -> Package Manager Console**.
2. Tại ô **Default project**, chọn **CMS.Data**.
3. Gõ lệnh sau và nhấn Enter: `Add-Migration InitialCreate` (Để lập bản vẽ thiết kế).



```
Package Manager Console  
Package source: All [v] [g] Default project: CMS.Data [v]  
PM> Add-Migration InitialCreate  
Build started...  
Build succeeded.  
To undo this action, use Remove-Migration.  
PM> |
```

4. Tiếp tục gõ lệnh: `Update-Database` (Để chính thức xây nhà - tạo Database).

TM PHẦN 4: THAY THẾ DỮ LIỆU GIẢ BẰNG DỮ LIỆU THẬT

quay lại các Controller đã làm ở Buổi 1 và sửa code:

Bây giờ, ta không dùng **List** giả nữa mà gọi trực tiếp từ Database.

Sửa file **CategoryController.cs**:

```
public class CategoryController : Controller {
    private readonly ApplicationDbContext _context;

    // "Tiêm" kết nối vào Controller
    public CategoryController(ApplicationDbContext context) {
        _context = context;
    }

    public IActionResult Index() {
        // Lấy dữ liệu THẬT từ bảng Categories trong SQL
        var data = _context.Categories.ToList();
        return View(data);
    }
}
```



KẾT QUẢ CẦN ĐẠT (CHECKPOINT)

1. Mở phần mềm SQL Server Management Studio (SSMS).
2. Kiểm tra thấy Database **ThaiCMS_DB** đã xuất hiện.
3. Chuột phải vào bảng **Categories** -> Edit Top 200 Rows -> Nhập tay vài dòng (Ví dụ: Tin Công Nghệ, Tin Giáo Dục, Tin Thời sự).

Results		Messages	
	Id	Name	Description
1	1	Tin Công Nghệ	Review Laptop, AI, IT
2	2	Giáo Dục	Tuyển sinh đại học 2026
3	3	Thời Sự	Tin thời sự trong nước, ngoài nước

4. Chạy trang web, vào **/Category**. Nếu dữ liệu bạn vừa nhập hiện lên màn hình → Bạn đã thành công 100%!

←	→	↺	localhost:7111/Category
🔍	Google Dịch	_HIU	ChatGPT
📁	master	Phim-hay	YouTube
📁	tools	test	Build_website
📁	_quantrong	_hoctap	
CMS.Backend	Home	Privacy	

DANH SÁCH DANH MỤC (DỮ LIỆU THẬT LẤY TỪ DATABASE)

ID	Tên danh mục	Mô tả
1	Tin Công Nghệ	Review Laptop, AI, IT
2	Giáo Dục	Tuyển sinh đại học 2026
3	Thời Sự	Tin thời sự trong nước, ngoài nước

CÁC ĐIỂM CẦN LƯU Ý (TIPS)

- **Kiểm tra SQL Server:** Nhớ đảm bảo **SQL Server Management Studio (SSMS)** đã hoạt động trước khi chạy lệnh Update-Database.

HƯỚNG DẪN KIỂM TRA SQL SERVER TRƯỚC KHI CHẠY MIGRATION

Bước 1: Kiểm tra Dịch vụ (Service) đang chạy

Nhiều trường hợp phần mềm SSMS mở được nhưng "máy chủ ngầm" (Service) bên dưới chưa chạy, dẫn đến lỗi kết nối.

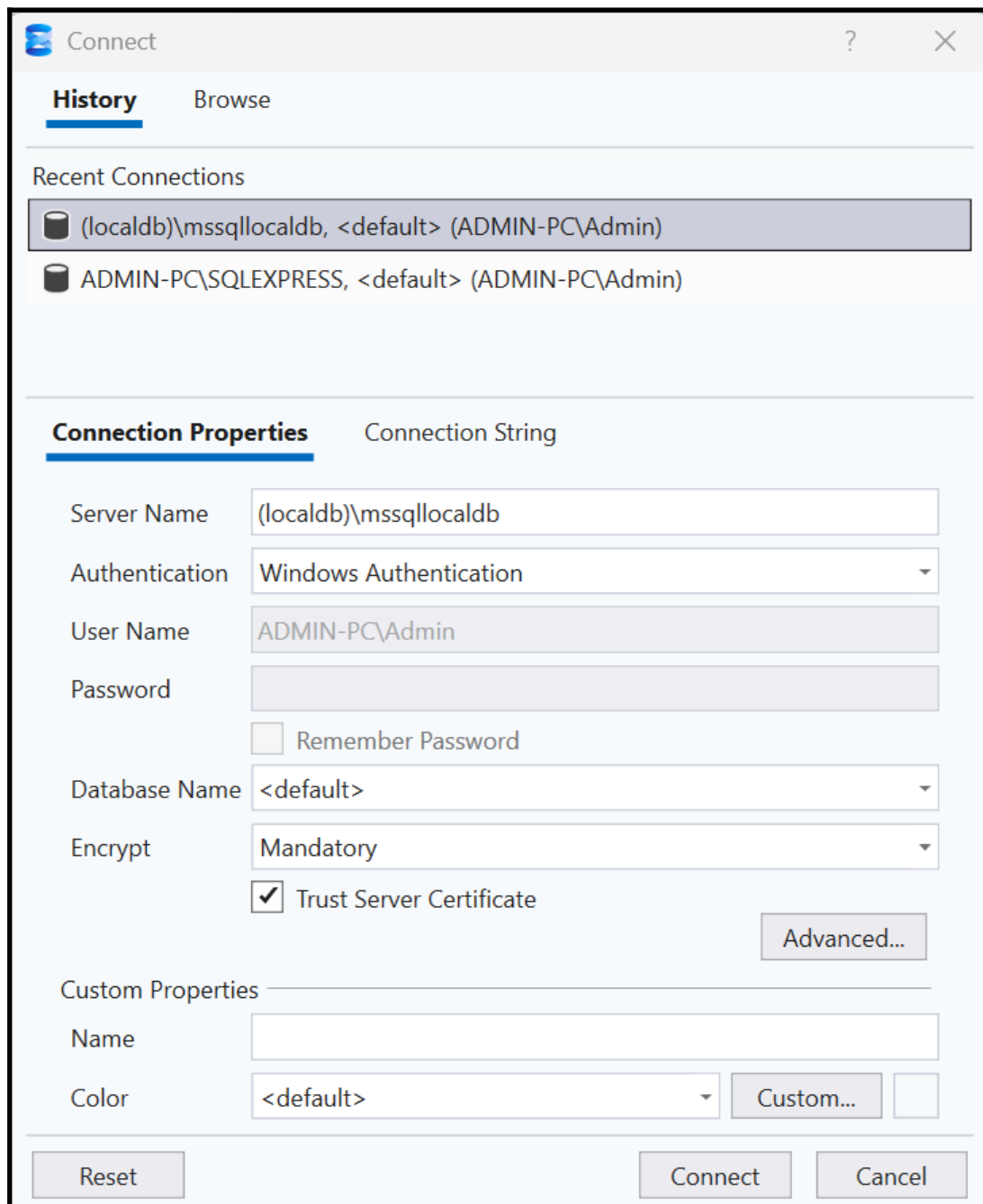
1. Nhấn phím **Windows + R**, gõ **services.msc** và nhấn Enter.
2. Tìm dòng có tên **SQL Server (MSSQLSERVER)** hoặc **SQL Server (SQLEXPRESS)**.
3. **Trạng thái (Status):** Phải là **Running**.
 - **Nếu trạng tron:** Chuột phải vào nó và chọn **Start**.

Bước 2: Đăng nhập thử bằng SSMS

Đây là bước xác nhận tên Server (Server Name) chính xác nhất để điền vào chuỗi kết nối.

1. Mở phần mềm **SQL Server Management Studio (SSMS)**.
2. Tại bảng hiện ra (Connect to Server):
 - **Server type:** Database Engine.
 - **Server name:** Thường là dấu chấm **.** hoặc **(localdb)\mssqllocaldb**.
 - **Authentication:** Windows Authentication.
3. Nhấn nút **Connect**.

4. **Kết quả:** Nếu bạn nhìn thấy danh sách thư mục (Databases, Security...) ở bên trái là OK.



Bước 3: Kiểm tra quyền tạo Database

Để lệnh **Update-Database** chạy được, tài khoản Windows phải có quyền tạo file.

- Trong SSMS, nhấn nút **New Query** trên thanh công cụ.

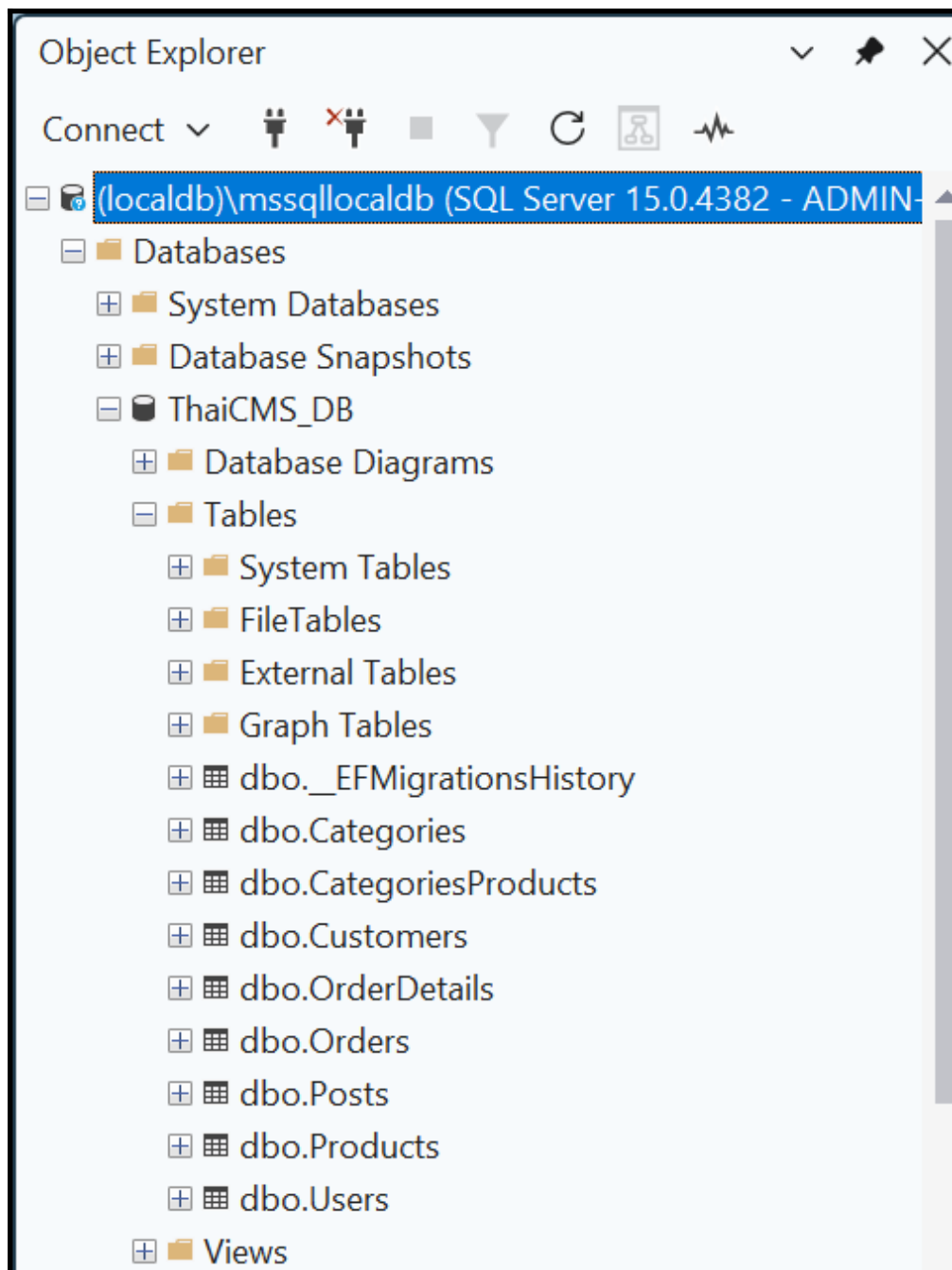
Gõ dòng lệnh sau và nhấn **F5**:

```
SELECT name FROM sys.databases
```

- Nếu hiện ra danh sách các Database mặc định (master, tempdb...), nghĩa là kết nối đã thông suốt và sẵn sàng để EF Core "đề" thêm Database mới.
 - **Lỗi chuỗi kết nối:** thường sai ở tên Server (Ví dụ: . hoặc (localdb)\mssqllocaldb). Bạn nên chuẩn bị sẵn 1-2 mẫu chuỗi kết nối chuẩn cho chúng ta.
 - **Giải thích về Dependency Injection (DI):** Đây là khái niệm khó của Chương 2. Bạn chỉ cần giải thích đơn giản: "Hệ thống tự động rút kết nối Database vào Controller cho chúng ta dùng, không cần khởi tạo thủ công".
-

KẾT QUẢ CẦN ĐẠT CUỐI BUỔI 2

1. mở SQL Server thấy xuất hiện Database tên là ThaiCMS_DB.



- Trong DB có đủ 8 bảng: *Categories*, *CategoriesProducts*, *Customers*, *OrderDetails*, *Orders*, *Posts*, *Products*, *Users*.
- Tự tay nhập 1-2 dòng dữ liệu vào SQL, sau đó chạy trang Web và thấy dữ liệu đó hiện lên màn hình.

PHẦN BÀI TẬP RÈN LUYỆN: TỰ LÀM CHỦ DỮ LIỆU VỚI POST VÀ USER

Yêu cầu: Sau khi đã hoàn thành kết nối bảng *Category*, hãy thực hiện tương tự cho hai thực thể còn lại là bài viết (*Post*) và người dùng (*User*).

BÀI TẬP 1: QUẢN LÝ BÀI VIẾT (TABLE POST)

Bước 1: Kiểm tra thực thể (Entity)

Đảm bảo trong Project **CMS.Data**, file **Post.cs** đã được định nghĩa đầy đủ các trường (Title, Content, CreatedDate, CategoryId...).

Bước 2: Nạp dữ liệu mẫu

- Mở **SSMS**, chuột phải vào bảng **Posts** -> **Edit Top 200 Rows**.
- Nhập ít nhất 5 bài viết mẫu (Lưu ý: Cột **CategoryId** phải điền số ID đã có sẵn trong bảng **Categories**).

Results		Messages				
	Id	Title	Content	ImageUrl	CreatedDate	CategoryId
1	2	Lợi ích của AI trong giáo dục	Khám phá cách Trí tuệ nhân tạo thay đổi cách họ...	/img/ai-edu.jpg	2026-04-01 00:00:00.0000000	1
2	6	5 thói quen tốt cho sức khỏe	Uống đủ nước và tập thể dục mỗi ngày giúp bạn...	/img/suc-khoe-001.jpg	2026-04-02 00:00:00.0000000	3

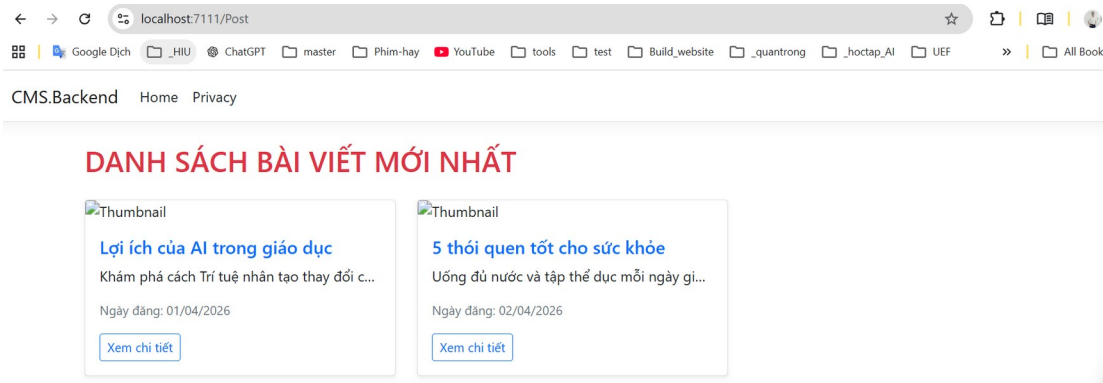
Bước 3: Tạo Controller và hiển thị dữ liệu

- Tạo file **PostController.cs** trong Project **CMS.Backend**.
- Thực hiện "Tiêm" **ApplicationDbContext** vào Controller (Constructor Injection).

Viết Action **Index()** để lấy danh sách bài viết từ Database:

```
public IActionResult Index() {  
    var posts = _context.Posts.ToList(); // Lấy tất cả bài viết  
    return View(posts);  
}
```

- Tạo View **Index.cshtml** để hiển thị danh sách dưới dạng bảng (Table) hoặc danh sách thẻ (Card).



BÀI TẬP 2: QUẢN LÝ NGƯỜI DÙNG (TABLE USER)

Bước 1: Nạp dữ liệu mẫu

- Trong **SSMS**, nhập ít nhất 2 người dùng mẫu vào bảng **Users**.
- Ví dụ:

Results		Messages			
	Id	Username	PasswordHash	FullName	Role
1	1	admin	123	Nguyen Cao Thai	Administrator
2	2	nhanvien001	456	Ly Phuong Uyen	Editor

Bước 2: Tạo chức năng xem danh sách thành viên

- Tạo **UserController.cs**.
- Viết Action **Index()** để lấy danh sách **Users**.
- Tạo View hiển thị thông tin thành viên (Lưu ý: Không nên hiển thị mật khẩu trên giao diện).

←

→

🔍

localhost:7111/User

☆

🔄

📄

📁

🔍

Google Dịch

📁 _HIU

🌐 ChatGPT

📁 master

📁 Phim-hay

📁 YouTube

📁 tools

📁 test

📁 Build_website

📁 _quantrong

📁 _hocapt_AI

📁 UEF

⋮

📁

📁

CMS.Backend

Home

Privacy

QUẢN LÝ THÀNH VIÊN

+ Thêm thành viên mới

ID	Tên Đăng Nhập	Họ Và Tên	Quyền Hạn	Thao Tác	
1	admin	Nguyen Cao Thai	Administrator	Sửa	Xóa
2	nhanvien001	Ly Phuong Uyen	Editor	Sửa	Xóa

Bảng dữ liệu mẫu cho tự nhập vào database

Table Category

Name	Description
Tin tức Công nghệ	Cập nhật xu hướng AI, IoT và lập trình.
Đời sống du lịch	Kinh nghiệm phượt và các điểm đến hấp dẫn.
Sức khỏe Thể thao	Các bài tập và chế độ ăn uống lành mạnh.
Giáo dục Kỹ năng	Phương pháp học tập và kỹ năng mềm .

Góc lập trình viên	Tài liệu ASP.NET Core và SQL Server.
--------------------	--------------------------------------

Table Post

Title	Content	ImageUrl	CategoryId	CreatedDate
Lộ trình học ASP.NET	Hướng dẫn chi tiết cho người mới bắt đầu...	/img/dotnet.jpg	5	2026-04-01
Top 5 bãi biển đẹp	Những địa điểm không thể bỏ qua mùa hè này...	/img/beach.jpg	2	2026-04-02
Chạy bộ đúng cách	Lợi ích tuyệt vời của việc chạy bộ mỗi sáng...	/img/run.jpg	3	2026-04-03
AI và tương lai	Trí tuệ nhân tạo đang thay đổi cuộc sống...	/img/ai.jpg	1	2026-04-04
Kỹ năng Teamwork	Cách phối hợp hiệu quả trong nhóm đồ án...	/img/team.jpg	4	2026-04-05

Table User

Username	PasswordHash	FullName	Role
admin	123456	Quản trị viên hệ thống	Admin
thai_gv	thai1969	Nguyễn Cao Thái	Editor
sv_01	student1	Nguyễn Văn A	User
sv_02	student2	Trần Thị B	User
moderator	mod789	Lê Văn C	Moderator

 **BÀI TẬP 3: Liệt kê danh sách (TABLE CategoriesProducts)**

 **BÀI TẬP 4: Liệt kê danh sách (TABLE Products)**

 **BÀI TẬP 5: Liệt kê danh sách (TABLE Customers)**

 **BÀI TẬP 6: Liệt kê danh sách (TABLE Orders)**

 **BÀI TẬP 7: Liệt kê danh sách (TABLE OrderDetails)**

BUỔI 3: TRUY VẤN LINQ & THAO TÁC DỮ LIỆU CHUYÊN SÂU

Mục tiêu bài học:

Sau khi kết thúc buổi học này, có khả năng:

- **Về kiến thức truy vấn (LINQ):**
 - Thành thạo cách sử dụng cú pháp LINQ để truy xuất dữ liệu từ SQL Server thông qua EF Core.
 - Hiểu và vận dụng được các hàm lọc (**Where**), sắp xếp (**OrderBy**), và lấy phần tử (**FirstOrDefault**).
 - Nắm vững kỹ thuật **Eager Loading** với lệnh **.Include()** để xử lý bài toán liên kết bảng (Join), tránh lỗi dữ liệu rỗng (null).
- **Về kỹ năng thao tác dữ liệu (CRUD):**
 - Xây dựng hoàn chỉnh luồng xử lý **Thêm - Xóa - Sửa** cho thực thể **Category** (Danh mục).
 - Phân biệt được sự khác nhau giữa phương thức **GET** (hiển thị giao diện) và **POST** (xử lý lưu dữ liệu) trong Controller.
 - Thực hiện thành thạo quy trình 2 bước để ghi dữ liệu: **Add/Update/Remove** (Thao tác trên bộ nhớ) và **SaveChanges** (Ghi xuống cơ sở dữ liệu).

1. ÔN TẬP & CHUẨN BỊ (5 PHÚT)

Ở Buổi 2, chúng ta đã dùng `_context.Categories.ToList()` để lấy dữ liệu thật. Tuy nhiên, trong thực tế, chúng ta không bao giờ lấy hết dữ liệu mà phải lọc, sắp xếp hoặc kết hợp các bảng.

2. TRUY VẤN DỮ LIỆU VỚI LINQ (CẦM TAY CHỈ VIỆC)

2.1. Lọc danh mục theo tham số (where) và sắp xếp (orderby)

Tình huống: Thay vì chỉ lấy bài viết của một chuyên mục cố định, chúng ta sẽ viết một hàm nhận vào một **id** bất kỳ. Khi người dùng truyền **id** nào, hệ thống sẽ lọc ra bài viết của chuyên mục đó và sắp xếp theo ngày đăng mới nhất.

A. Mã nguồn trong Controller (**PostController.cs**)

Sửa hàm **Index** (hoặc tạo hàm mới tên là **ListByCategory**) như sau:

```
// Tham số 'id' được truyền vào từ URL (ví dụ: /Post/Index/5)
public IActionResult Index(int? id)
{
    // 1. Kiểm tra nếu không có id truyền vào thì trả về lỗi hoặc toàn bộ bài viết
    if (id == null)
    {
        return BadRequest("Vui lòng cung cấp mã danh mục.");
    }
}
```

```
// 2. Sử dụng LINQ với tham số 'id' linh hoạt
var posts = _context.Posts
    .Where(p => p.CategoryId == id)
    .OrderByDescending(p => p.CreatedDate)
    .Include(p => p.Category)
    .ToList();

// 3. Truyền dữ liệu ra View
return View(posts);
}
```

B. Giải thích chi tiết

- **Tham số `int? id`:** Dấu chấm hỏi `?` cho phép `id` có thể bị rỗng (null) nếu người dùng không gõ số ID trên URL. Điều này giúp chương trình không bị lỗi "văng" ngay lập tức.
- **`.Where(p => p.CategoryId == id)`:** Đây là biểu thức Lambda. Hệ thống sẽ quét bảng `Posts` và chỉ chọn những dòng nào có cột `CategoryId` khớp chính xác với con số mà truyền vào từ trình duyệt.
- **`.OrderByDescending`:** Giúp bài viết vừa đăng xong sẽ hiện lên vị trí đầu tiên, giống như các trang báo điện tử hiện nay.

Tổng quát: công thức "vàng" này để áp dụng cho mọi bảng dữ liệu:

```
Biến_Kết_Quả = _context.Tên_Bảng
    .Hàm_Lọc
    .Hàm_Sắp_Xếp
    .Hàm_Liên_Kết.Thực_Thi();
```

Giải thích chi tiết các "Mắt xích"

01. **`_context.Posts`:** Điểm bắt đầu. Nó đại diện cho bảng dữ liệu trong SQL Server mà đã cấu hình ở **Buổi 2**.
02. **`.Where(p => ...)`:** Màn lọc dữ liệu.
 - a. `p` đại diện cho một dòng trong bảng (Post).
 - b. `p.CategoryId == id` là điều kiện lọc: "Chỉ lấy những bài viết có mã danh mục bằng với tham số truyền vào".
03. **`.OrderByDescending(p => ...)`:** Bộ sắp xếp.
 - a. Dùng để đưa dữ liệu mới nhất (ngày lớn nhất) lên đầu trang.
04. **`.Include(p => p.Category)`:** Lệnh nạp chồng (Join bảng).
 - a. Trong tài liệu anh soạn, bảng `Post` có liên kết với `Category`.
 - b. Lệnh này bắt buộc phải có để lấy được **Tên danh mục**, nếu không thuộc tính này sẽ bị `null` và gây lỗi trang web.
05. **`.ToList()`:** Lệnh chốt hạ.
 - a. Nếu không có lệnh này, C# vẫn chưa thực sự tải dữ liệu về. Nó chỉ chính thức truy vấn SQL khi gặp lệnh `.ToList()`.

C. Cách chạy thử để kiểm tra (Testing)

cần hiểu cách truyền tham số trên thanh địa chỉ trình duyệt:

1. **Trường hợp 1:** Gõ `https://localhost:xxxx/Post/Index/5`
 - Hệ thống sẽ hiểu `id = 5` và lọc ra các bài viết thuộc "Góc lập trình viên".
2. **Trường hợp 2:** Gõ `https://localhost:xxxx/Post/Index/2`
 - Hệ thống sẽ hiểu `id = 2` và lọc ra các bài viết thuộc "Đời sống du lịch".

D. Lưu ý quan trọng (Tips)

"Khi dùng tham số, chúng ta phải đảm bảo con số `id` đó **đã tồn tại** trong bảng `Categories`. Nếu truyền vào một số không có trong Database (ví dụ số 999), danh sách trả về sẽ trống trơn (không có bài viết nào) nhưng chương trình vẫn chạy đúng."

2.2. Kỹ thuật Join bảng (Include) - CỰC KỲ QUAN TRỌNG

Lỗi thường gặp: Khi hiển thị bài viết, tên danh mục bị trống (null). **Giải pháp:** Dùng `.Include()` để yêu cầu EF Core lấy kèm dữ liệu từ bảng liên quan.

```
// Lấy bài viết kèm theo thông tin của Danh mục
var posts = _context.Posts
    .Include(p => p.Category)
    .ToList();
```

Giải thích: Lệnh này giúp có thể gọi `@item.Category.Name` ngoài View mà không bị lỗi.vào trong thẻ

Cập nhật lại code file `Index.cshtml` trong thư mục `View/Post`, bổ sung thêm dòng code:
`<span class="badge bg-info text-dark">@item.Category.Name</span>`

trong thẻ `<div class="card-body">` như sau:

```
@model IEnumerable<CMS.Data.Entities.Post>

<div class="container mt-4">
    <h2 class="mb-4 text-danger">DANH SÁCH BÀI VIẾT MỚI NHẤT</h2>
    <div class="row">
        @foreach (var item in Model)
        {
            <div class="col-md-4 mb-3">
                <div class="card shadow-sm">
                    
                    <div class="card-body">
                        <span class="badge bg-info
text-dark">@item.Category.Name</span>
                        <h5 class="card-title text-primary">@item.Title</h5>
                        <p class="card-text text-truncate">@item.Content</p>
                        <p class="text-muted small">Ngày đăng:
@item.CreatedDate.ToString("dd/MM/yyyy")</p>
```

```

<a href="/Post/Details/@item.Id" class="btn btn-outline-primary btn-sm">Xem chi tiết</a>
</div>
</div>
</div>
}
</div>
</div>

```

Chạy lại trang **http://localhost:xxxx/Post**

DANH SÁCH BÀI VIẾT

⊕ Thêm bài viết mới



Tin Công Nghệ
Lợi ích của AI trong giáo dục
 Khám phá cách Trí tuệ nhân tạo thay đổi c...
 Ngày đăng: 13/04/2026
[Xem chi tiết](#) [Sửa](#) [Xóa](#)



Sức Khỏe
Chỉ số Tiểu đường tuýp 2.
 Tiểu đường tuýp 2. Tiểu đường tuýp 2. Tiế...
 Ngày đăng: 13/04/2026
[Xem chi tiết](#) [Sửa](#) [Xóa](#)



Tin Công Nghệ
Lập trình .NET Core cho người mới
 Hướng dẫn từng bước xây dựng ứng dụng...
 Ngày đăng: 13/04/2026
[Xem chi tiết](#) [Sửa](#) [Xóa](#)



3. THAO TÁC DỮ LIỆU CHUYÊN SÂU (CRUD)

3.1. Tìm kiếm và hiển thị chi tiết (Details)

Cập nhật Action **Details** trong **PostController.cs**:

```

// GET: Post/Details/5
public IActionResult Details(int id)
{
    // 1. Truy vấn bài viết theo ID
    // Sử dụng .Include(p => p.Category) để lấy kèm thông tin Danh mục (Join bảng)
    var post = _context.Posts
        .Include(p => p.Category)
        .FirstOrDefault(p => p.Id == id);

    // 2. Kiểm tra nếu không tìm thấy bài viết (tránh lỗi màn hình trắng)
    if (post == null)
    {
        return NotFound(); // Trả về trang lỗi 404
    }

    // 3. Truyền dữ liệu sang View
    return View(post);
}

```

```
}
```

3.2. Thêm mới dữ liệu (Create)

Mục tiêu của phần này là giúp nắm vững quy trình **gửi dữ liệu từ trình duyệt về máy chủ** và **ghi dữ liệu đó vào Database**. Chúng ta tạm thời chưa quan tâm đến việc người dùng nhập đúng hay sai (phần này sẽ học ở Buổi 5).

A. Code xử lý tại Controller (CategoryController.cs)

Trong Controller, chúng ta cần 2 hàm (Action). Một hàm để **mở trang nhập liệu** và một hàm để **thực hiện việc lưu**.

C#

```
// 1. Hàm GET: Dùng để hiển thị giao diện Form cho nhập
[HttpGet]
public IActionResult Create()
{
    return View();
}

// 2. Hàm POST: Dùng để đón dữ liệu từ Form gửi lên và lưu vào SQL
[HttpPost]
public IActionResult Create(Category model)
{
    // BƯỚC 1: Thêm dữ liệu vào bộ nhớ tạm của Entity Framework
    _context.Categories.Add(model);

    // BƯỚC 2: Ra lệnh cho hệ thống ghi dữ liệu thật sự vào SQL Server
    _context.SaveChanges();

    // Sau khi lưu thành công, tự động quay về trang danh sách
    return RedirectToAction("Index");
}
```

B. Giải thích quy trình 2 bước (Bắt buộc phải nhớ)

hãy nhấn mạnh rằng thiếu 1 trong 2 bước này, dữ liệu sẽ không bao giờ xuất hiện trong Database:

1. **Lệnh `_context.Categories.Add(model)`**: Đây là bước "Đăng ký". Dữ liệu từ các ô nhập liệu được gom lại thành một đối tượng `Category` và đưa vào danh sách chờ của hệ thống.
2. **Lệnh `_context.SaveChanges()`**: Đây là bước "Chốt đơn". Chỉ khi lệnh này chạy, Entity Framework mới tạo ra câu lệnh SQL `INSERT INTO...` để đẩy dữ liệu vào bảng `Categories` trong SQL Server.

C. Code giao diện nhập liệu (Create.cshtml)

tạo file `Create.cshtml` bên trong thư mục `Views/Category` và dán đoạn code sau:

Razor CSHTML

```
@model CMS.Data.Entities.Category
```

```
<div class="container mt-4">
  <h2 class="text-primary">THÊM DANH MỤC MỚI</h2>
  <hr />

  <form asp-action="Create" method="post">
    <div class="mb-3">
      <label class="form-label">Tên danh mục:</label>
      <input asp-for="Name" class="form-control" />
    </div>

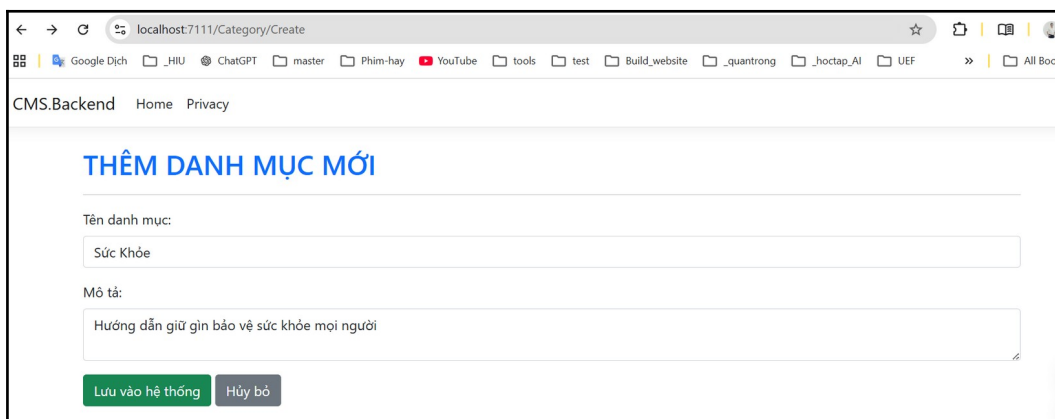
    <div class="mb-3">
      <label class="form-label">Mô tả:</label>
      <textarea asp-for="Description" class="form-control"></textarea>
    </div>

    <button type="submit" class="btn btn-success">Lưu vào hệ thống</button>
    <a asp-action="Index" class="btn btn-secondary">Hủy bỏ</a>
  </form>
</div>
```

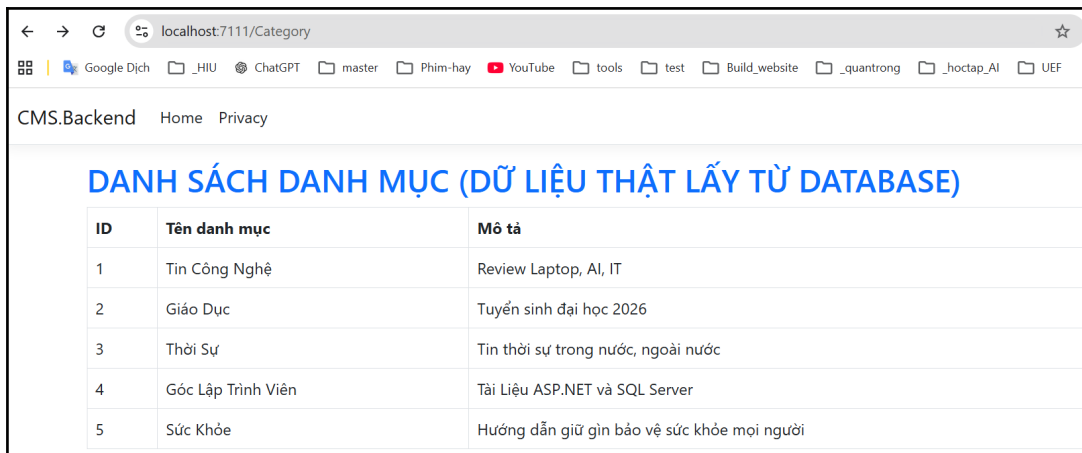
D. Ví dụ thực hành tại lớp

Yêu cầu thực hiện theo đúng thứ tự:

1. **Chạy dự án (F5)** và truy cập đường dẫn: <https://localhost:xxxx/Category/Create>.
2. **Nhập dữ liệu mẫu:**
 - Tên danh mục: Sức Khỏe
 - Mô tả: Hướng dẫn giữ gìn bảo vệ sức khỏe mọi người



3. **Nhấn nút "Lưu vào hệ thống".**



ID	Tên danh mục	Mô tả
1	Tin Công Nghệ	Review Laptop, AI, IT
2	Giáo Dục	Tuyển sinh đại học 2026
3	Thời Sự	Tin thời sự trong nước, ngoài nước
4	Góc Lập Trình Viên	Tài Liệu ASP.NET và SQL Server
5	Sức Khỏe	Hướng dẫn giữ gìn bảo vệ sức khỏe mọi người

4. **Kiểm chứng:** Quay lại phần mềm **SSMS**, chuột phải vào bảng **Categories** chọn **Select Top 1000 Rows** để thấy dòng dữ liệu vừa nhập đã nằm trong Database.

Thêm nút lệnh “Thêm danh mục mới” trên đầu trang View/Category/Index.cshtml với code sau:

```
<a asp-action="Create" class="btn btn-success">
    <i class="bi bi-plus-circle"></i> Thêm danh mục mới
</a>
```

Giải thích:

- **asp-action="Create"**: Đây là Tag Helper sẽ tự động tạo đường dẫn đến trang thêm mới (/Category/Create)
- **btn-success**: Tạo màu xanh lá cây đặc
- **<i class="bi bi-plus-circle"></i>** hiển thị biểu tượng dấu cộng trong vòng tròn



ID	Tên danh mục	Mô tả	Thao tác
1	Tin Công Nghệ	Review Laptop, AI, IT	<button>Sửa</button> <button>Xóa</button>
2	Giáo Dục	Tuyển sinh đại học 2026	<button>Sửa</button> <button>Xóa</button>
4	Góc Lập Trình Viên	Tài Liệu ASP.NET và SQL Server	<button>Sửa</button> <button>Xóa</button>
6	Sức Khỏe	Tăng cường sức khỏe mọi người	<button>Sửa</button> <button>Xóa</button>

📌 **Ghi chú:** "Ở buổi này, nếu không nhập gì mà nhấn Lưu, hệ thống vẫn sẽ tạo ra một dòng trống trong Database. Đừng lo lắng, chúng ta sẽ học cách ngăn chặn việc này ở bài học về **Validation** (Buổi 5)!".

3.3 Xóa 1 danh mục trong Category

ID	Tên danh mục	Mô tả	Thao tác
1	Tin Công Nghệ	Review Laptop, AI, IT	Sửa Xóa
2	Giáo Dục	Tuyển sinh đại học 2026	Sửa Xóa
3	Thời Sự	Tin thời sự trong nước, ngoài nước	Sửa Xóa
4	Góc Lập Trình Viên	Tài Liệu ASP.NET và SQL Server	Sửa Xóa
5	Sức Khỏe	Hướng dẫn giữ gìn bảo vệ sức khỏe mọi người	Sửa Xóa

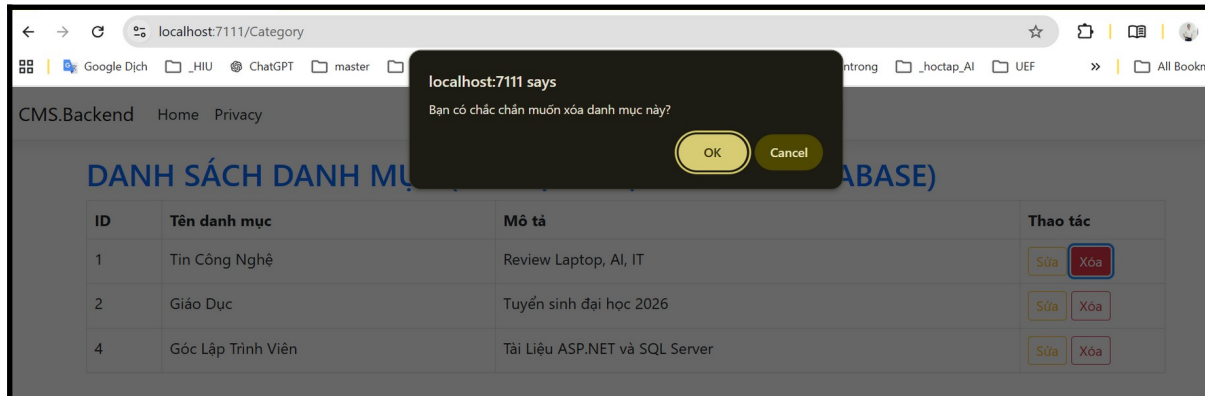
Chỉnh code View/Category/Index.cshtml

```
@model IEnumerable<CMS.Data.Entities.Category>
<h2 class="text-primary">DANH SÁCH DANH MỤC (DỮ LIỆU THẬT LẤY TỪ
DATABASE)</h2>
<table class="table table-bordered">
  <thead>
    <tr>
      <th>ID</th>
      <th>Tên danh mục</th>
      <th>Mô tả</th>
      <th style="width:150px">Thao tác</th>
    </tr>
  </thead>
  <tbody>
    @foreach (var item in Model)
    {
      <tr>
        <td>@item.Id</td>
        <td>@item.Name</td>
        <td>@item.Description</td>
        <td>
          <a asp-action="Edit" asp-route-id="@item.Id"
            class="btn btn-sm btn-outline-warning">Sửa</a>

          <a asp-action="Delete" asp-route-id="@item.Id"
            class="btn btn-sm btn-outline-danger"
            onclick="return confirm('Bạn có chắc chắn muốn xóa danh mục
            này?')">Xóa</a>
        </td>
      </tr>
    }
  </tbody>
</table>
```

Giải thích: ý nghĩa các thẻ "Helper" để không gõ nhầm:

- **asp-action="Edit"**: Chỉ định khi nhấn nút, trình duyệt sẽ tìm đến hàm (Action) tên là **Edit** trong Controller.
- **asp-route-id="@item.Id"**: Đây là cách ASP.NET Core tự động đính kèm ID vào URL (Ví dụ: `/Category/Edit/1`).
- **onclick="return confirm(...)"**: Đây là đoạn mã Javascript nhỏ để hiện bảng hỏi xác nhận, tránh việc lỡ tay xóa mất dữ liệu quý giá trong Database.



Tiếp theo ta cần code để thực hiện xóa 1 danh mục trong database

Trong ASP.NET Core MVC, quy trình xóa diễn ra qua 3 bước: **Tìm đối tượng** -> **Xóa khỏi bộ nhớ tạm** -> **Lưu thay đổi vào CSDL**.

A. Mã nguồn xử lý tại **CategoryController.cs**

Hướng dẫn thêm Action sau vào Controller:

C#

```
// Action nhận vào Id của danh mục cần xóa
public IActionResult Delete(int id)
{
    // Bước 1: Tìm đối tượng danh mục trong Database bằng Id
    var category = _context.Categories.Find(id);

    // Kiểm tra nếu tìm thấy thì mới xóa
    if (category != null)
    {
        // Bước 2: Lệnh xóa khỏi bộ nhớ tạm (Tracking)
        _context.Categories.Remove(category);

        // Bước 3: Chốt phiên làm việc, xóa thực sự trong SQL Server
        _context.SaveChanges();
    }

    // Sau khi xóa xong, quay lại trang danh sách để cập nhật giao diện
    return RedirectToAction("Index");
}
```

B. Giải thích các câu lệnh

- **_context.Categories.Find(id)**: Trước khi xóa, hệ thống phải "cầm" được đối tượng đó trong tay. Lệnh này tìm kiếm theo khóa chính (Id).

- **`_context.Categories.Remove(category)`**: Lệnh này chỉ mới đánh dấu đối tượng là "Sẽ bị xóa". Dữ liệu trong SQL Server lúc này vẫn còn nguyên.
- **`_context.SaveChanges()`**: Đây là lệnh quan trọng nhất. EF Core sẽ tạo ra câu lệnh SQL `DELETE FROM Categories WHERE Id = ...` và gửi xuống máy chủ SQL để xóa thực sự.

C. Ví dụ thực hành kiểm chứng (Lab 3)

Thực hiện thao tác sau để thấy kết quả:

1. **Mở trang danh sách**: Truy cập <https://localhost:xxxx/Category>.
2. **Thực hiện xóa**: Nhấn nút **Xóa** ở dòng danh mục số 3 (Thời Sự).
3. **Xác nhận**: Nhấn **OK** khi bảng hỏi xác nhận hiện lên.

Lưu ý quan trọng về Ràng buộc toàn vẹn dữ liệu:

"Nếu xóa một Danh mục (**Category**) mà danh mục đó đang có các Bài viết (**Post**) bên trong, hệ thống sẽ báo lỗi. Vì theo logic, không thể có bài viết mà không có danh mục quản lý. Cần xóa hết bài viết thuộc danh mục đó trước khi xóa danh mục cha."

3.4. THỰC HIỆN CHỈNH SỬA DỮ LIỆU (UPDATE)

Quy trình chỉnh sửa gồm 2 giai đoạn:

1. **Giai đoạn 1 (GET)**: Tìm danh mục theo ID và hiện thông tin lên Form để sửa.
2. **Giai đoạn 2 (POST)**: Nhận dữ liệu đã sửa từ Form và cập nhật vào SQL Server.

A. Mã nguồn xử lý tại `CategoryController.cs`

hướng dẫn thêm 2 hàm sau vào Controller:

C#

```
// 1. Hàm GET: Tìm dữ liệu cũ và đổ lên Form
[HttpGet]
public IActionResult Edit(int id)
{
    // Tìm danh mục trong Database theo Id [cite: 348, 350]
    var category = _context.Categories.Find(id);

    if (category == null) return NotFound();

    return View(category); // Gửi đối tượng tìm được sang giao diện Edit
}

// 2. Hàm POST: Nhận dữ liệu mới từ người dùng và lưu lại
```

```
[HttpPost]
public IActionResult Edit(Category model)
{
    // Lệnh cập nhật đối tượng vào bộ nhớ tạm
    _context.Categories.Update(model);

    // Lưu thay đổi thực sự xuống SQL Server [cite: 504, 509]
    _context.SaveChanges();

    // Quay lại trang danh sách để xem kết quả
    return RedirectToAction("Index");
}
```

B. Giải thích các lệnh chính

- `_context.Categories.Find(id)`: Dùng để lấy đúng dòng dữ liệu cần sửa từ Database lên bộ nhớ.
- `_context.Categories.Update(model)`: Entity Framework sẽ tự động so sánh dữ liệu cũ và mới để chuẩn bị câu lệnh SQL UPDATE.
- `return View(category)`: Giúp các ô nhập liệu trên trang Sửa tự động hiển thị tên và mô tả hiện tại của danh mục đó.

C. Tạo giao diện chỉnh sửa (Edit.cshtml)

← → ↻ 🌐 localhost:7111/Category/Edit/1

🗂️ | 🌐 Google Dịch 📁 _HIU 🌀 ChatGPT 📁 master 📁 Phim-hay 📁 YouTube 📁 tools 📁 test 📁 B

CMS.Backend Home Privacy

CHỈNH SỬA DANH MỤC

Tên danh mục:

Tin Công Nghệ

Mô tả:

Review Laptop, AI, IT

Cập nhật thay đổi Hủy bỏ

© 2026 - CMS.Backend - [Privacy](#)

Sinh viên tạo file `Edit.cshtml` trong thư mục `Views/Category` với nội dung tương tự trang Create nhưng có thêm **Hidden Field** để giữ ID:

Razor CSHTML

```
@model CMS.Data.Entities.Category
```

```
<div class="container mt-4">  
  <h2 class="text-warning">CHỈNH SỬA DANH MỤC</h2>  
  <hr />
```

```
<form asp-action="Edit" method="post">  
  <input type="hidden" asp-for="Id" />
```

```
<div class="mb-3">  
  <label class="form-label fw-bold">Tên danh mục:</label>  
  <input asp-for="Name" class="form-control" />  
</div>
```

```
<div class="mb-3">  
  <label class="form-label fw-bold">Mô tả:</label>  
  <textarea asp-for="Description" class="form-control" rows="3"></textarea>  
</div>
```

```
<button type="submit" class="btn btn-warning">Cập nhật thay đổi</button>
<a asp-action="Index" class="btn btn-secondary">Hủy bỏ</a>
</form>
</div>
```

D. Thực hành kiểm chứng (Lab 3)

Thực hiện thao tác sau:

1. Truy cập trang danh sách: `https://localhost:xxxx/Category`.
2. Nhấn nút **Sửa** ở dòng "Tin Công Nghệ".
3. Thay đổi tên thành: Tin Công Nghệ & AI.
4. Nhấn **Cập nhật thay đổi**.
5. **Kết quả:** Hệ thống quay về trang danh sách và tên danh mục đã được thay đổi thành công.

📌 **Lưu ý :** “Tuyệt đối không được quên dòng `<input type="hidden" asp-for="Id" />`. Nếu thiếu dòng này, khi nhấn Lưu, hệ thống sẽ không biết đang muốn sửa dòng nào và có thể gây ra lỗi tạo mới nhầm lẫn!”.

4. DỮ LIỆU MẪU THỰC HÀNH (LAB 3)

Để kiểm chứng các câu lệnh LINQ vừa học, cần có dữ liệu thật trong Database `ThaiCMS_DB`. Hãy mở **SSMS**, chuột phải vào từng bảng và nhập đúng các dòng sau:

4.1. Bảng Danh mục (Table Category)

(Nhập vào bảng `dbo.Categories`)

Id	Name	Description
1	Tin tức Công nghệ	Cập nhật xu hướng AI, IoT và lập trình.
2	Đời sống du lịch	Kinh nghiệm phượt và các điểm đến hấp dẫn.
5	Góc lập trình viên	Tài liệu ASP.NET Core và SQL Server.

4.2. Bảng Bài viết (Table Post)

(Nhập vào bảng `dbo.Posts` - Lưu ý cột `CategoryId` phải khớp với ID ở bảng trên)

Title	Content	CategoryId	CreatedDate
Lộ trình học ASP.NET	Hướng dẫn chi tiết cho người mới bắt đầu...	5	2026-04-01

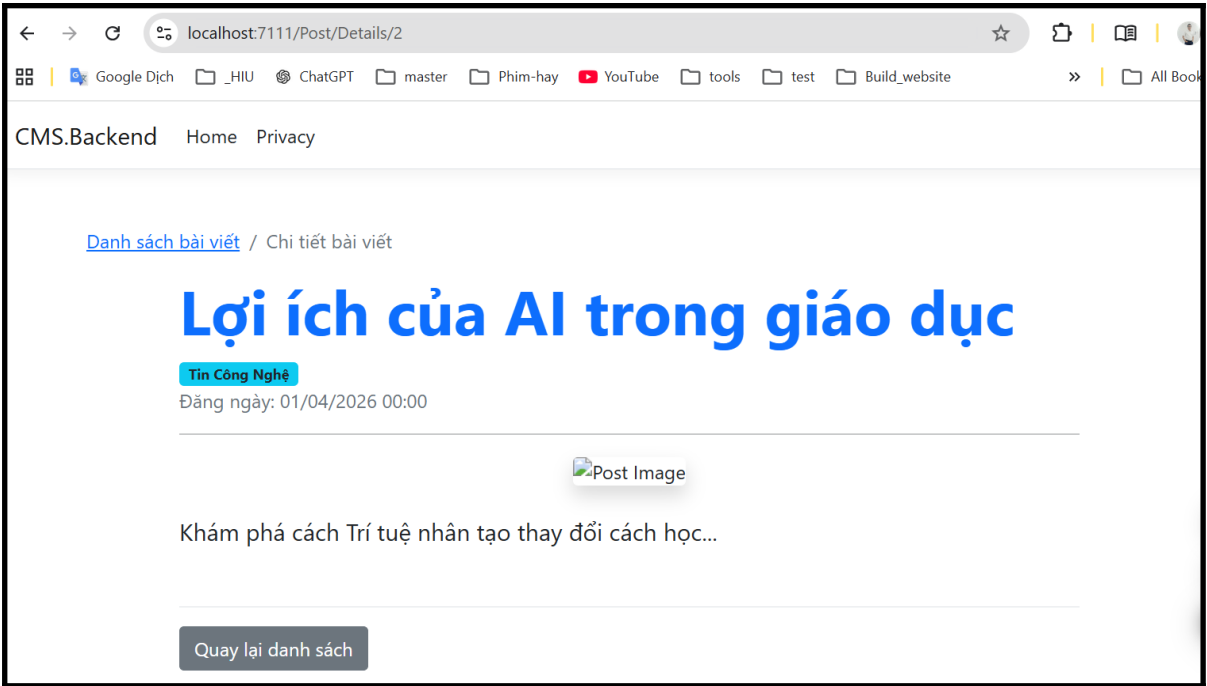
Top 5 bãi biển đẹp	Những địa điểm không thể bỏ qua mùa hè này...	2	2026-04-02
AI và tương lai	Trí tuệ nhân tạo đang thay đổi cuộc sống...	1	2026-04-04

4.3. Bảng Người dùng (Table User)
(Nhập vào bảng `dbo.Users`)

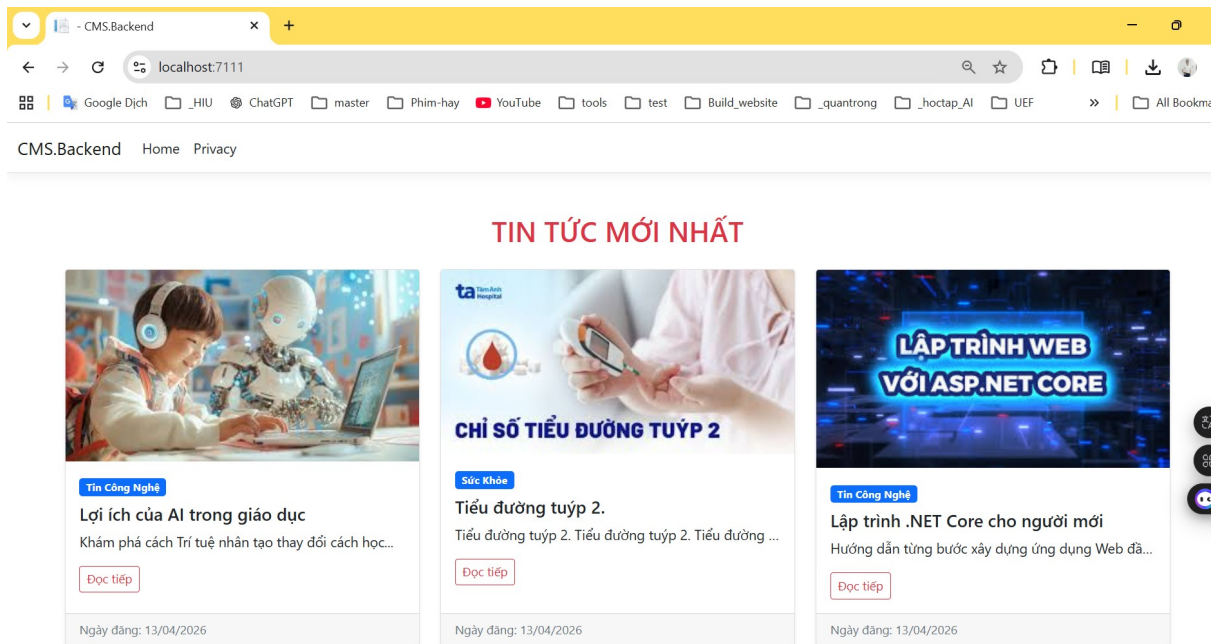
Username	FullName	Role	PasswordHash
admin	Quản trị viên hệ thống	Admin	123456
thai_gv	Nguyễn Cao Thái	Editor	thai1969

5. BÀI TẬP TỰ RÈN LUYỆN (CHALLENGE)

Yêu cầu 1: Tại trang quản lý bài viết (`Post/Details.cshtml`), hãy hiển thị thêm "Tên danh mục" bằng cách sử dụng `.Include()` trong Controller.



Yêu cầu 2: HIỂN THỊ 3 BẢN TIN MỚI NHẤT LÊN TRANG CHỦ
Chúng ta sẽ kết hợp các hàm LINQ đã học là `OrderByDescending` và `Take`.



Mục tiêu là lọc từ cơ sở dữ liệu ra 3 bài viết có ngày đăng (`CreatedDate`) gần đây nhất để trình bày lên trang chủ.

Bước 1: Chỉnh sửa Controller (`Controllers/HomeController.cs`)

Mở `HomeController.cs`, thêm `ApplicationContext` vào và viết logic lọc dữ liệu:

C#

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using CMS.Data; // Thư mục chứa DbContext [cite: 568]
using System.Linq;

public class HomeController : Controller
{
    private readonly ApplicationDbContext _context;

    public HomeController(ApplicationDbContext context)
    {
        _context = context;
    }

    public IActionResult Index()
    {
        // LINQ: Lấy 3 bài viết mới nhất
        var latestPosts = _context.Posts
            [cite_start].Include(p => p.Category) // Lấy kèm tên danh mục để hiển thị
            [cite_start].OrderByDescending(p => p.CreatedDate) // Sắp xếp ngày
            mới nhất lên đầu
            .Take(3) // Chỉ lấy đúng 3 bản tin đầu tiên
            .ToList();

        return View(latestPosts);
    }
}
```

```
}
}
```

Bước 2: Giải thích cú pháp LINQ

- **.OrderByDescending(p => p.CreatedDate)**: Sắp xếp toàn bộ bài viết theo thứ tự thời gian giảm dần (mới nhất đứng trước).
- **.Take(3)**: Đây là lệnh "cắt" lấy đúng 3 dòng đầu tiên của danh sách đã sắp xếp.
- **.Include(p => p.Category)**: Đảm bảo lấy được tên danh mục (ví dụ: "Tin Công Nghệ") để hiển thị trên thẻ bài viết.

Bước 3: Cập nhật giao diện trang chủ (Views/Home/Index.cshtml)

Sử dụng vòng lặp `@foreach` để hiển thị danh sách 3 bài viết dưới dạng thẻ (Card):

Razor CSHTML

```
@model IEnumerable<CMS.Data.Entities.Post>

<div class="container mt-5">
  <h2 class="text-center mb-4 text-danger">TIN TỨC MỚI NHẤT</h2>
  <div class="row">
    @foreach (var item in Model)
    {
      <div class="col-md-4">
        <div class="card shadow-sm h-100">
          
          <div class="card-body">
            <span class="badge bg-primary mb-2">@item.Category.Name</span>
            <h5 class="card-title">@item.Title</h5>
            <p class="card-text text-truncate">@item.Content</p>
            <a href="/Post/Details/@item.Id" class="btn btn-outline-danger btn-sm">Đọc tiếp</a>
          </div>
          <div class="card-footer text-muted small">
            Ngày đăng: @item.CreatedDate.ToString("dd/MM/yyyy")
          </div>
        </div>
      </div>
    }
  </div>
</div>
```

Kết quả cần đạt: Khi chạy trang chủ (`localhost:xxxx/`), phải thấy đúng 3 bài viết mới nhất hiện lên đẹp mắt.

CMS.Backend

localhost:7111

Google Dịch _HIU ChatGPT master Phim-hay YouTube tools test Build_website _quantrong _hocktap_AI UEF All Book

CMS.Backend Home Privacy

TIN TỨC MỚI NHẤT



Tin Công Nghệ

Lợi ích của AI trong giáo dục

Khám phá cách Trí tuệ nhân tạo thay đổi cách học...

[Đọc tiếp](#)

Ngày đăng: 13/04/2026



Sức Khỏe

Tiểu đường tuýp 2.

Tiểu đường tuýp 2. Tiểu đường tuýp 2. Tiểu đường ...

[Đọc tiếp](#)

Ngày đăng: 13/04/2026



Tin Công Nghệ

Lập trình .NET Core cho người mới

Hướng dẫn từng bước xây dựng ứng dụng Web đã...

[Đọc tiếp](#)

Ngày đăng: 13/04/2026

BUỔI 4: XÂY DỰNG GIAO DIỆN QUẢN TRỊ (ADMIN PANEL) TOÀN DIỆN

Mục tiêu của chúng ta:

- Thiết lập hệ thống Layout chung chuyên biệt cho khu vực quản trị.
- Xây dựng Sidebar điều hướng thông minh cho cả 3 chức năng: Danh mục, Bài viết và Thành viên.
- Hoàn thiện các trang danh sách (Index) và biểu mẫu (Form) theo đúng chuẩn MVC.

1. THIẾT KẾ SIDEBAR VÀ LAYOUT ADMIN (_LayoutAdmin.cshtml)

Chúng ta sẽ xây dựng một bộ khung riêng biệt để người quản trị dễ dàng thao tác.

A. Cấu trúc Sidebar bên trái

Thanh Menu này là "xương sống" để chúng ta di chuyển giữa các chức năng:

Liên kết	Controller tương ứng	Công dụng / Chức năng
Bảng điều khiển	Home/Dashboard	Xem tổng quan số lượng bài viết, thành viên.
Quản lý Danh mục	Category/Index	Thêm/Sửa/Xóa các nhóm tin (Công nghệ, Du lịch...).
Quản lý Bài viết	Post/Index	Biên tập nội dung, đăng ảnh và chọn danh mục bài viết.
Quản lý Thành viên	User/Index	Quản lý tài khoản và phân quyền (Admin/Editor).

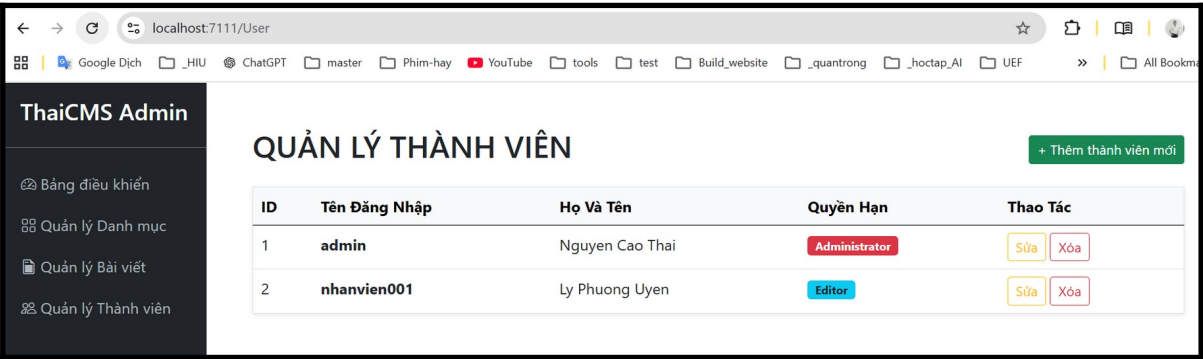
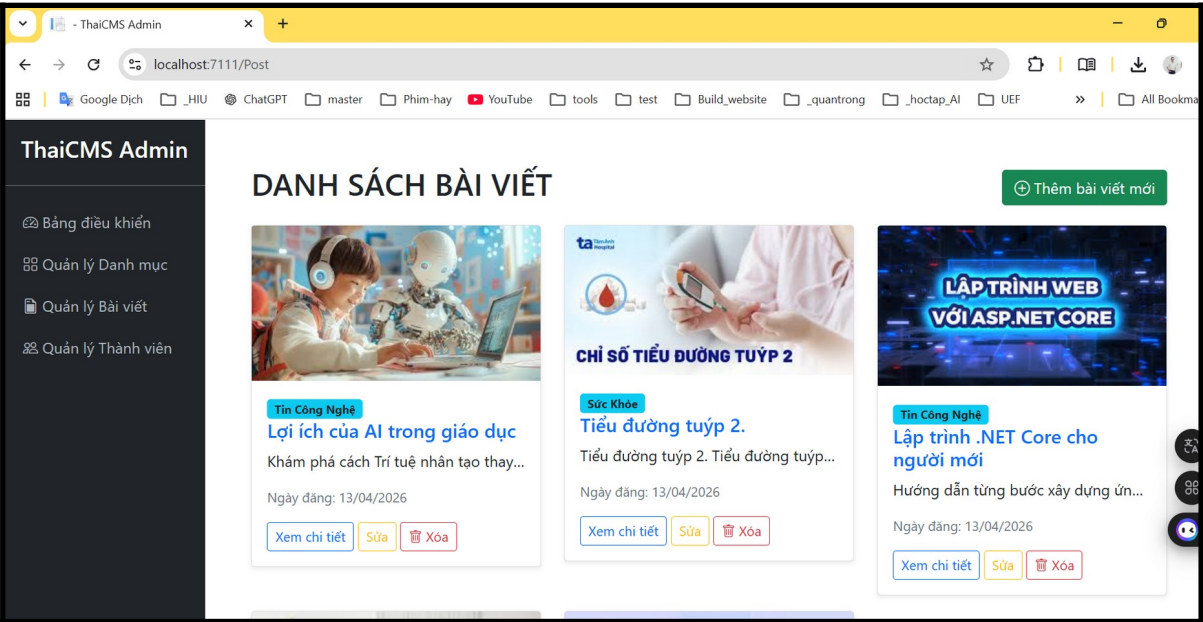
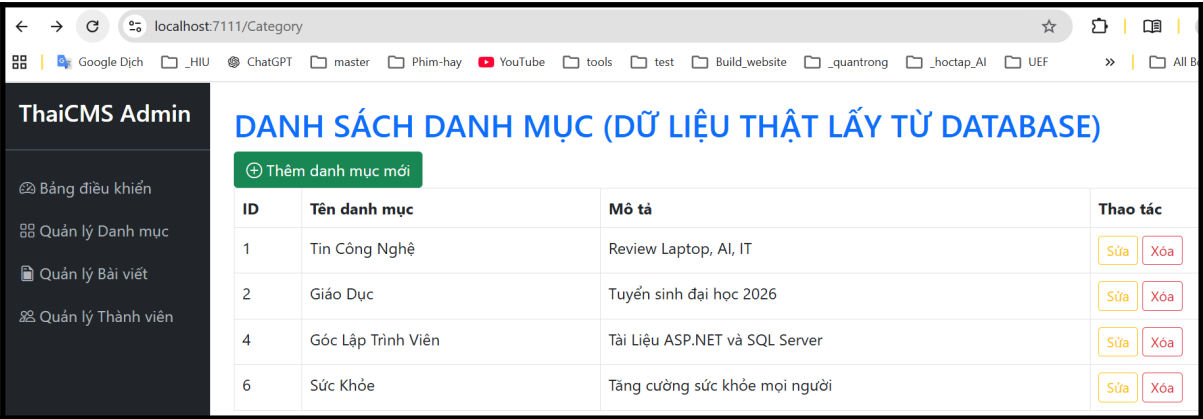
B. Thực hành tạo Layout

Trong thư mục Views/Shared, chúng ta tạo file _LayoutAdmin.cshtml. Chúng ta sử dụng Bootstrap để chia cột:

- Cột trái (col-md-3):** Chứa Sidebar cố định với các thẻ `<a asp-controller="..." asp-action="Index">`.
- Cột phải (col-md-9):** Nơi hiển thị nội dung chính thông qua lệnh `@RenderBody()`.

HƯỚNG DẪN TẠO LAYOUT QUẢN TRỊ (_LayoutAdmin.cshtml)

Mục tiêu của chúng ta: Tạo ra một giao diện chuyên nghiệp có thanh điều hướng bên trái (Sidebar) và vùng nội dung bên phải, giúp chúng ta quản lý dự án một cách khoa học.



a. CHUẨN BỊ FILE

- Chúng ta vào thư mục **CMS.Backend**.
- Tìm đến thư mục **Views/Shared**.
- Chuột phải chọn **Add -> New Item -> Razor Layout** và đặt tên là LayoutAdmin.cshtml.

b. NỘI DUNG MÃ NGUỒN (CODE CHI TIẾT)

Chúng ta sử dụng thư viện Bootstrap để chia khung giao diện. Hãy chép đoạn mã sau vào file vừa tạo:

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewData["Title"] - ThaiCMS Admin</title>
  <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-icons.css">
  <style>
    .sidebar { min-height: 100vh; background-color: #212529; color: white; }
    .sidebar a { color: #adb5bd; text-decoration: none; display: block; padding: 10px 20px; }
    .sidebar a:hover { background-color: #343a40; color: white; }
    .sidebar .active { background-color: #0d6efd; color: white; }
  </style>
</head>
<body>
  <div class="container-fluid">
    <div class="row">
      <nav class="col-md-3 col-lg-2 sidebar p-0">
        <div class="p-3 text-center border-bottom border-secondary">
          <h4 class="text-white">ThaiCMS Admin</h4>
        </div>
        <div class="mt-3">
          <a asp-controller="Home" asp-action="Index">
            <i class="bi bi-speedometer2"></i> Bảng điều khiển
          </a>
          <a asp-controller="Category" asp-action="Index">
            <i class="bi bi-grid"></i> Quản lý Danh mục
          </a>
        </div>
      </nav>
    </div>
  </div>
```

```
<a asp-controller="Post" asp-action="Index">
    <i class="bi bi-file-earmark-post"></i> Quản lý Bài viết
</a>

<a asp-controller="User" asp-action="Index">
    <i class="bi bi-people"></i> Quản lý Thành viên
</a>

<a asp-controller="ProductCategory" asp-action="Index">
    <i class="bi bi-grid"></i> Danh mục sản phẩm
</a>

<a asp-controller="Product" asp-action="Index">
    <i class="bi bi-box-seam"></i> Sản phẩm
</a>

<a asp-controller="Customer" asp-action="Index">
    <i class="bi bi-people"></i> Khách hàng
</a>

<a asp-controller="Order" asp-action="Index">
    <i class="bi bi-grid"></i> Đơn hàng
</a>

<a asp-controller="OrderDetail" asp-action="Index">
    <i class="bi bi-grid"></i> Đơn hàng chi tiết
</a>

</div>

</nav>

<main class="col-md-9 col-lg-10 ms-sm-auto p-4">
    @RenderBody() </main>
</div>
</div>

<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
@await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

c. GIẢI THÍCH CHI TIẾT CHO CHÚNG TA

- **Thanh Sidebar (Cột trái):** Chúng ta sử dụng `col-md-3` để tạo thực đơn cố định. Tại đây, các thẻ `<a>` kết hợp với **Tag Helpers** (`asp-controller`, `asp-action`) giúp chúng ta di chuyển nhanh chóng giữa 3 bảng: Category, Post và User.
- **Lệnh `@RenderBody()`:** Đây là "trạm chờ" dữ liệu. Khi chúng ta mở trang Quản lý Bài viết, toàn bộ code của trang đó sẽ được nạp vào đúng vị trí này trong Layout.
- **Tag Helpers (`asp-controller`):** Giúp chúng ta không cần gõ đường dẫn thủ công, tránh lỗi "link chết" khi thay đổi tên thư mục.

d. CÁCH ÁP DỤNG LAYOUT NÀY VÀO CÁC TRANG

Để các trang như `Index.cshtml` của Post hay Category nhận Layout mới này, chúng ta cần khai báo dòng sau ở đầu mỗi file View:

Razor CSHTML

```
@{  
    Layout = "_LayoutAdmin";  
}
```

e. CÔNG DỤNG CỦA CÁC LIÊN KẾT TRÊN SIDEBAR

Chức năng	Ý nghĩa đối với chúng ta
Bảng điều khiển	Giúp chúng ta xem nhanh tình trạng website.
Quản lý Danh mục	Giúp chúng ta tổ chức các nhóm tin tức (Công nghệ, Du lịch...).
Quản lý Bài viết	Đây là nơi chúng ta biên tập nội dung chính cho các bài viết.
Quản lý Thành viên	Giúp chúng ta kiểm soát ai có quyền sửa bài, ai chỉ có quyền xem.

Lời nhắn của chúng ta: "Khi chúng ta hoàn thiện file Layout này, việc quản lý 3 bảng dữ liệu sẽ trở nên cực kỳ dễ dàng và đồng bộ. Hãy chú ý kiểm tra các biểu tượng `bi-` (Bootstrap Icons) để giao diện Admin của chúng ta trông thật chuyên nghiệp!"

2. QUẢN TRỊ BÀI VIẾT (POST)

Đây là phần quan trọng nhất vì bài viết có mối liên kết với danh mục.

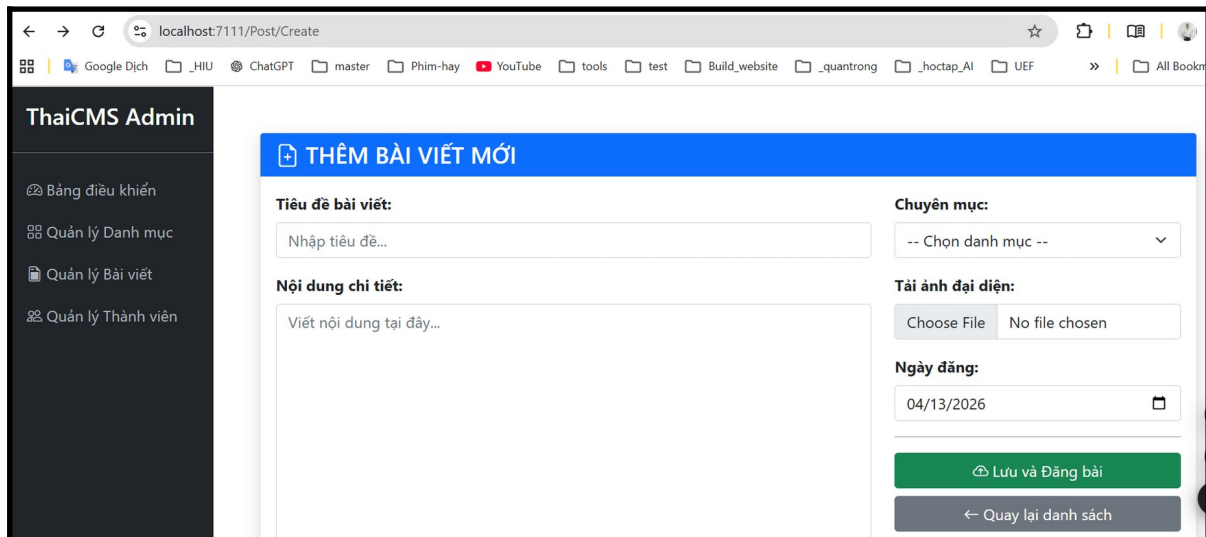
Trước tiên cập nhật code trang View/Post/Index.cshtml có thêm nút lệnh “**Thêm bài viết mới**”

```
@model IEnumerable<CMS.Data.Entities.Post>
@{
    Layout = "_LayoutAdmin";
}

<div class="container mt-4">
    <div class="d-flex justify-content-between align-items-center mb-3">
        <h2 class="text-dark">DANH SÁCH BÀI VIẾT</h2>
        <a asp-action="Create" class="btn btn-success">
            <i class="bi bi-plus-circle"></i> Thêm bài viết mới
        </a>
    </div>
    <div class="row">
        @foreach (var item in Model)
        {
            <div class="col-md-4 mb-3">
                <div class="card shadow-sm">
                    
                    <div class="card-body">
                        <span class="badge bg-info text-dark">@item.Category.Name</span>
                        <h5 class="card-title text-primary">@item.Title</h5>
                        <p class="card-text text-truncate">@item.Content</p>
                        <p class="text-muted small">Ngày đăng:
                            @item.CreatedDate.ToString("dd/MM/yyyy")</p>
                        <a href="/Post/Details/@item.Id" class="btn btn-outline-primary btn-sm">Xem chi tiết</a>
                        <a asp-action="Edit" asp-route-id="@item.Id" class="btn btn-sm btn-outline-warning">
                            Sửa
                        </a>
                        <a asp-action="Delete" asp-route-id="@item.Id"
                            class="btn btn-sm btn-outline-danger"
                            onclick="return confirm('Chúng ta có chắc chắn muốn xóa bài viết này không?')">
                            <i class="bi bi-trash"></i> Xóa
                        </a>
                    </div>
                </div>
            </div>
        }
    </div>
</div>
```

```
</div>
</div>
```

1) Nhập mới bài viết (Post)



A. Chuẩn bị dữ liệu tại **PostController.cs**

Chúng ta cần lấy danh sách Danh mục từ Database để người dùng chọn khi viết bài.

C#

```
// 1. Hàm hiển thị form tạo mới bài viết (GET)
[HttpGet]
public IActionResult Create()
{
    // Chúng ta lấy danh sách Category để đổ vào ViewBag
    ViewBag.CategoryList = new SelectList(_context.Categories, "Id", "Name");
    return View();
}

[HttpPost]
public IActionResult Create(Post model, IFormFile uploadImage)
{
    if (uploadImage != null && uploadImage.Length > 0)
    {
        // 1. Định nghĩa đường dẫn lưu file: wwwroot/uploads
        string folder = Path.Combine(Directory.GetCurrentDirectory(), "wwwroot",
        "uploads");

        // Tạo thư mục nếu chưa tồn tại
        if (!Directory.Exists(folder)) Directory.CreateDirectory(folder);
    }
}
```

```
// 2. Tạo tên file duy nhất để không bị đè dữ liệu
string fileName = Guid.NewGuid().ToString() +
Path.GetExtension(uploadImage.FileName);
string filePath = Path.Combine(folder, fileName);

// 3. Chép file vào thư mục
using (var stream = new FileStream(filePath, FileMode.Create))
{
    uploadImage.CopyTo(stream);
}

// 4. Lưu đường dẫn vào CSDL để sau này hiển thị
model.ImageUrl = "/uploads/" + fileName;
}

_context.Posts.Add(model);
_context.SaveChanges();
return RedirectToAction("Index");
}
```

Giải thích:

1. HÀM HIỂN THỊ FORM (GET METHOD)

Hàm này có nhiệm vụ chuẩn bị "nguyên liệu" trước khi giao diện hiện ra.

- **_context.Categories**: Chúng ta truy cập vào bảng Danh mục trong Database.
- **new SelectList(..., "Id", "Name")**: Chúng ta tạo ra một danh sách chọn. Trong đó, **Id** là giá trị sẽ lưu xuống Database, còn **Name** là tên danh mục để **chúng ta** nhìn thấy trên màn hình.
- **ViewBag.CategoryList**: Chúng ta bỏ danh sách này vào một "chiếc túi" tạm thời để gửi sang View.

2. HÀM LƯU DỮ LIỆU VÀ XỬ LÝ FILE (POST METHOD)

Đây là nơi xử lý logic chính khi người dùng nhấn nút "Lưu".

A. Tiếp nhận dữ liệu

- **Post model**: Chứa các thông tin dạng chữ (Tiêu đề, Nội dung).
- **IFormFile uploadImage**: Đây là đối tượng đặc biệt đại diện cho file ảnh mà **chúng ta** chọn từ máy tính.

B. Quy trình xử lý File (4 bước "vàng")

1. **Xác định vị trí lưu**: **Path.Combine(..., "wwwroot", "uploads")** giúp chúng ta trở đúng vào thư mục chứa ảnh của website. Nếu thư mục **uploads** chưa có, lệnh **Directory.CreateDirectory** sẽ tự động tạo ra nó.
2. **Định danh file duy nhất**: **Guid.NewGuid().ToString()** tạo ra một chuỗi ký tự ngẫu nhiên (ví dụ: **7c578a...**). Việc này giúp tránh tình trạng hai người cùng upload ảnh tên **anh1.jpg** gây đè dữ liệu của nhau.

3. **Sao chép file vào máy chủ:** `uploadImage.CopyTo(stream)` thực hiện việc chép dữ liệu từ trình duyệt của người dùng vào thư mục trên máy chủ của **chúng ta**.
4. **Lưu đường dẫn:** Thay vì lưu cả tấm ảnh vào Database (làm chậm hệ thống), **chúng ta** chỉ lưu cái "địa chỉ" của nó (ví dụ: `/uploads/abc.jpg`) vào thuộc tính `model.ImageUrl`.

C. Ghi dữ liệu vào Database

- `_context.Posts.Add(model)`: Chúng ta đưa thông tin bài viết vào danh sách chờ.
- `_context.SaveChanges()`: Đây là lệnh "chốt đơn". Chỉ khi lệnh này chạy, dữ liệu mới thực sự được ghi xuống SQL Server.
- `RedirectToAction("Index")`: Sau khi xong việc, **chúng ta** tự động chuyển hướng người dùng quay lại trang danh sách bài viết để họ thấy kết quả.

B. Giao diện biểu mẫu (`Create.cshtml`)

Chúng ta sử dụng thẻ `<select>` kết hợp Tag Helper để tạo ô chọn chuyên mục.

```
@model CMS.Data.Entities.Post
```

```
@{
    Layout = "_LayoutAdmin";
    ViewData["Title"] = "Thêm bài viết mới";
}
```

```
<div class="container mt-4">
    <div class="card shadow">
        <div class="card-header bg-primary text-white">
            <h4 class="mb-0"><i class="bi bi-file-earmark-plus"></i> THÊM BÀI VIẾT MỚI</h4>
        </div>
        <div class="card-body">
            <form asp-action="Create" method="post">
                <div class="row">
                    <div class="col-md-8">
                        <div class="mb-3">
                            <label class="form-label fw-bold">Tiêu đề bài viết:</label>
                            <input asp-for="Title" class="form-control" placeholder="Nhập tiêu đề..." />
                        </div>

                        <div class="mb-3">
                            <label class="form-label fw-bold">Nội dung chi tiết:</label>
                            <textarea asp-for="Content" class="form-control" rows="10" placeholder="Viết nội dung tại đây..."></textarea>
                        </div>
                    </div>
                    <div class="col-md-4">
                        <div class="mb-3">
```

```

        <label class="form-label fw-bold">Chuyên mục:</label>
        <select asp-for="CategoryId" class="form-select" asp-
items="ViewBag.CategoryList">
            <option value="">-- Chọn danh mục --</option>
        </select>
    </div>

    <div class="mb-3">
        <label class="form-label fw-bold">Link ảnh đại diện (URL):</label>
        <input asp-for="ImageUrl" class="form-control"
placeholder="https://..." />
    </div>

    <div class="mb-3">
        <label class="form-label fw-bold">Ngày đăng:</label>
        <input asp-for="CreatedDate" type="date" class="form-control"
value="@DateTime.Now.ToString("yyyy-MM-dd")" />
    </div>

    <hr />
    <div class="d-grid gap-2">
        <button type="submit" class="btn btn-success">
            <i class="bi bi-cloud-arrow-up"></i> Lưu và Đăng bài
        </button>
        <a asp-action="Index" class="btn btn-secondary">
            <i class="bi bi-arrow-left"></i> Quay lại danh sách
        </a>
    </div>
</div>
</div>
</div>
</div>
</div>
</div>

```

chý th, thêm vài bài vit mi, kim chng.

2) Chức năng xóa bài viết (Delete)

Đây là chức năng đơn giản nhất để **chúng ta** bắt đầu. Quy trình thực hiện gồm 3 bước: Tìm -> Xóa tạm -> Lưu thật.

A. Tại PostController.cs

Chúng ta viết Action Delete:

C#

```
public IActionResult Delete(int id)
{
    // 1. Tìm bài viết theo Id

    var post = _context.Posts.Find(id);

    if (post != null)
    {
        // 2. Xóa khỏi bộ nhớ tạm

        _context.Posts.Remove(post);

        // 3. Cập nhật xuống SQL Server

        _context.SaveChanges();
    }

    return RedirectToAction("Index");
}
```

B. Tại View (Index.cshtml)

Chúng ta thêm nút bấm kèm lời xác nhận để tránh **chúng ta** lỡ tay xóa nhầm:

Razor CSHTML

```
<a asp-action="Delete" asp-route-id="@item.Id"
    class="btn btn-sm btn-outline-danger"
    onclick="return confirm('Chúng ta có chắc chắn muốn xóa bài viết này không?')">
    <i class="bi bi-trash"></i> Xóa
</a>
```

3. Chức năng sửa bài viết (Edit)

Chức năng này phức tạp hơn vì **chúng ta** cần hiển thị lại dữ liệu cũ vào Form để người dùng chỉnh sửa.

A. Tại PostController.cs

Chúng ta cần 2 hàm: Một hàm để hiện dữ liệu cũ (GET) và một hàm để lưu thay đổi (POST).

C#

```
// GET: Hiển thị form kèm dữ liệu cũ
[HttpGet]
public IActionResult Edit(int id)
{
    var post = _context.Posts.Find(id);
    if (post == null) return NotFound();

    // Chuẩn bị lại danh sách danh mục để người dùng có thể đổi chuyên mục
    ViewBag.CategoryList = new SelectList(_context.Categories, "Id", "Name",
    post.CategoryId);
    return View(post);
}

// POST: Thực hiện cập nhật
[HttpPost]
public IActionResult Edit(Post model, IFormFile uploadImage)
{
    // Bước 1: Kiểm tra xem người dùng có chọn file ảnh mới không
    if (uploadImage != null && uploadImage.Length > 0)
    {
        // Thực hiện quy trình upload giống như trang Create
        string folder = Path.Combine(Directory.GetCurrentDirectory(), "wwwroot",
        "uploads");
        if (!Directory.Exists(folder)) Directory.CreateDirectory(folder);

        string fileName = Guid.NewGuid().ToString() +
        Path.GetExtension(uploadImage.FileName);
        string filePath = Path.Combine(folder, fileName);

        using (var stream = new FileStream(filePath, FileMode.Create))
        {
            uploadImage.CopyTo(stream);
        }

        // Cập nhật đường dẫn ảnh mới vào model
        model.ImageUrl = "/uploads/" + fileName;
    }
    else
    {
        // Bước quan trọng: Nếu không upload ảnh mới, chúng ta phải giữ lại ảnh cũ
        // Chúng ta cần lấy lại giá trị ImageUrl từ Database để tránh bị ghi đè thành rỗng
        var oldPost = _context.Posts.AsNoTracking().FirstOrDefault(p => p.Id == model.Id);
        if (oldPost != null && string.IsNullOrEmpty(model.ImageUrl))
        {
            model.ImageUrl = oldPost.ImageUrl;
        }
    }
    _context.Posts.Update(model);
}
```

```
_context.SaveChanges();  
return RedirectToAction("Index");  
}
```

B. Tại View (Edit.cshtml)

Giao diện trang Sửa gần như giống hệt trang `Create.cshtml`. Điểm khác biệt duy nhất là **chúng ta** cần thêm một thẻ ẩn (Hidden Input) để giữ lại `Id` của bài viết:

Razor CSHTML

```
<form asp-action="Edit" method="post">  
  
    <input type="hidden" asp-for="Id" />  
  
  
</form>
```

3. GIẢI THÍCH

- **Tại sao cần `input type="hidden"`?** Giải thích nếu không có thẻ ẩn này, khi nhấn Lưu, hệ thống sẽ không biết **chúng ta** đang sửa bài viết số mấy và sẽ báo lỗi hoặc tạo ra một bài viết mới tinh.
- **Lưu ý về hình ảnh:** Nếu bài viết cũ đã có ảnh, khi sửa **chúng ta** nên hiển thị ảnh đó lên để người dùng biết họ có cần thay ảnh mới hay không.
- **Thứ tự ưu tiên:** hãy nhắc **chúng ta** luôn làm chức năng Xóa trước vì nó giúp hiệu luồng `Find -> Remove -> SaveChanges` một cách nhanh nhất.

CÂU HỎI GỢI Ý ĐỂ CHÚNG TA THẢO LUẬN

1. Nếu chúng ta xóa một bài viết, tấm ảnh trong thư mục `uploads` có mất đi không? (Đáp án: Không, chúng ta chỉ mới xóa địa chỉ trong Database, muốn xóa file vật lý cần thêm lệnh `System.IO.File.Delete`).
2. Để nội dung bài viết không chỉ là những dòng văn bản thô (Plain Text) mà có thể định dạng được **đậm, nghiêng, chia danh mục hay chèn bảng**, **chúng ta** sẽ tích hợp một trình soạn thảo văn bản giàu tính năng (Rich Text Editor).

Phổ biến và dễ dùng nhất hiện nay cho ASP.NET Core là CKEditor 5.

1. TÍCH HỢP CKEDITOR VÀO TRANG CREATE.CSHTML

Thay vì phải cài đặt phức tạp, chúng ta sẽ sử dụng CDN (đường dẫn trực tuyến) để tích hợp nhanh vào dự án.

Bước 1: Thêm thư viện vào View

Anh hướng dẫn chúng ta mở file Views/Post/Create.cshtml, tìm đến phần @section Scripts ở cuối file và thêm đoạn mã sau:

Razor CSHTML

```
@section Scripts {  
    <script src="https://cdn.ckeditor.com/ckeditor5/39.0.1/classic/ckeditor.js"></script>  
  
    <script>  
        // 2. Kích hoạt trình soạn thảo cho thẻ textarea có id là 'Content'  
        ClassicEditor  
            .create(document.querySelector('#Content'))  
            .catch(error => {  
                console.error(error);  
            });  
    </script>  
}
```

Bước 2: Kiểm tra ID của thẻ Textarea

Chúng ta cần đảm bảo thẻ textarea dành cho nội dung bài viết có id khớp với đoạn Script trên (mặc định Tag Helper asp-for="Content" sẽ tự sinh ra id="Content").

Razor CSHTML

```
<div class="mb-3">  
    <label class="form-label fw-bold">Nội dung chi tiết:</label>  
    <textarea asp-for="Content" class="form-control" rows="10"></textarea>  
</div>
```

2. HIỂN THỊ NỘI DUNG RA NGOÀI (VIEW DETAILS/INDEX)

Một lưu ý cực kỳ quan trọng: Khi dùng trình soạn thảo, nội dung lưu vào Database sẽ có các thẻ HTML (ví dụ: <p>Chào anh</p>).

Nếu chúng ta hiển thị theo cách thông thường @item.Content, trình duyệt sẽ hiện luôn cả các chữ . Để hiển thị đúng định dạng (Render HTML), chúng ta dùng lệnh @Html.Raw():

Tại trang Views/Post/Details.cshtml hoặc Index.cshtml:

Razor CSHTML

```
<div class="post-content">  
    @Html.Raw(Model.Content)  
</div>
```

Làm sao để khi nhấn "Sửa", trình soạn thảo **CKEditor** hiện ra đúng nội dung cũ? (Đáp án: Chỉ cần đặt đúng ID cho textarea, CKEditor sẽ tự nạp nội dung từ asp-for="Content").

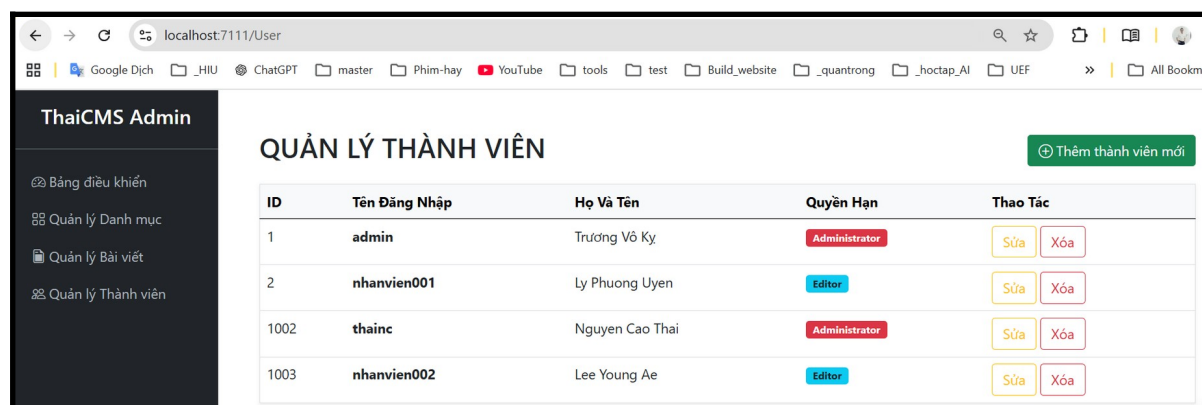
Bài tập tự thực hành:

Để thực hiện được phần **Login (Xác thực)** và **Phân quyền** ở Buổi 5, trang Quản trị User ở Buổi 4 cần phải đảm bảo lưu trữ được 2 thông tin cốt lõi: **Mật khẩu (Password)** và **Vai trò (Role)**.

Dưới đây là hướng dẫn bổ sung chi tiết để hoàn thiện chức năng Quản trị User chuẩn xác nhất.

PHẦN BỔ SUNG: QUẢN TRỊ THÀNH VIÊN HOÀN CHỈNH

Mục tiêu: Đảm bảo khi tạo mới User, dữ liệu phải có đủ Username, Password và Role để hệ thống Login có thể đối soát.

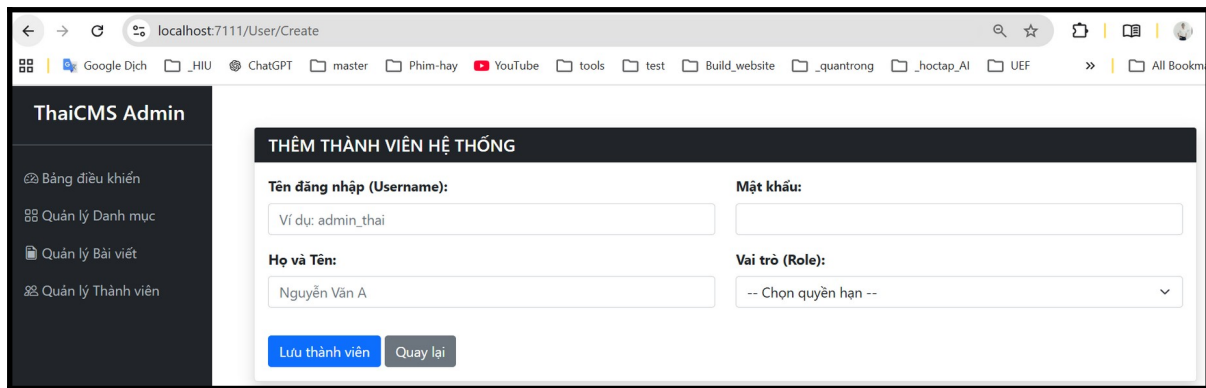


Đảm bảo có thêm code trong file View/User/Index.cshtml dùng gắn nút lệnh thêm thành viên mới trên trang quản trị thành viên

```
<div class="d-flex justify-content-between align-items-center mb-3">
  <h2 class="text-dark">QUẢN LÝ THÀNH VIÊN</h2>
  <a asp-action="Create" class="btn btn-success">
    <i class="bi bi-plus-circle"></i> Thêm thành viên mới
  </a>
</div>
```

1. CẬP NHẬT GIAO DIỆN THÊM MỚI (VIEWS/USER/CREATE.CSHTML)

Chúng ta cần một Form có ô nhập mật khẩu và một danh sách chọn (Dropdown) để phân quyền.



Thao tác: Tạo file `Create.cshtml` trong thư mục `Views/User`.

Razor CSHTML

@model CMS.Data.Entities.User

```
@{
    Layout = "_LayoutAdmin";
    ViewData["Title"] = "Thêm thành viên";
}
```

```
<div class="container mt-4">
    <div class="card shadow">
        <div class="card-header bg-dark text-white">
            <h5 class="mb-0">THÊM THÀNH VIÊN HỆ THỐNG</h5>
        </div>
        <div class="card-body">
            <form asp-action="Create" method="post">
                <div class="row">
                    <div class="col-md-6 mb-3">
                        <label class="form-label fw-bold">Tên đăng nhập (Username):</label>
                        <input asp-for="Username" class="form-control" placeholder="Ví dụ: admin_thai"
required />
                    </div>
                    <div class="col-md-6 mb-3">
                        <label class="form-label fw-bold">Mật khẩu:</label>
                        <input asp-for="PasswordHash" type="password" class="form-control" required />
                    </div>
                </div>
                <div class="row">
                    <div class="col-md-6 mb-3">
                        <label class="form-label fw-bold">Họ và Tên:</label>
                        <input asp-for="FullName" class="form-control" placeholder="Nguyễn Văn A"
required />
                    </div>
                    <div class="col-md-6 mb-3">
                        <label class="form-label fw-bold">Vai trò (Role):</label>
                        <select asp-for="Role" class="form-select" required>
                            <option value="">-- Chọn quyền hạn --</option>
                        </select>
                    </div>
                </div>
            </form>
        </div>
    </div>
</div>
```

```
        <option value="Admin">Quản trị viên (Admin)</option>
        <option value="Editor">Biên tập viên (Editor)</option>
    </select>
</div>
</div>

<div class="mt-3">
    <button type="submit" class="btn btn-primary">Lưu thành viên</button>
    <a asp-action="Index" class="btn btn-secondary">Quay lại</a>
</div>
</form>
</div>
</div>
</div>
```

2. XỬ LÝ LƯU DỮ LIỆU TẠI CONTROLLER (USERCONTROLLER.CS)

Chúng ta cần viết hàm POST để nhận dữ liệu từ Form và ghi xuống SQL Server.

Thao tác: Mở `UserController.cs` và thêm hàm sau:

C#

```
[HttpPost]
public IActionResult Create(User model)
{
    // Kiểm tra xem tên đăng nhập đã tồn tại chưa
    var checkExist = _context.Users.Any(u => u.Username == model.Username);
    if (checkExist)
    {
        ModelState.AddModelError("Username", "Tên đăng nhập này đã có người dùng!");
        return View(model);
    }

    // Lưu User mới vào Database
    _context.Users.Add(model);
    _context.SaveChanges();

    return RedirectToAction("Index");
}
```

3. LƯU Ý QUAN TRỌNG KHI THỰC HIỆN

1. **Trường Role (Vai trò):** Phải nhập chính xác chữ **"Admin"** hoặc **"Editor"** (phân biệt hoa thường tùy cấu hình). Buổi 5 sẽ dựa vào đúng chữ này để cho phép người dùng vào trang hay không.

2. **Trường PasswordHash:** Hiện tại ở Buổi 4, chúng ta đang lưu mật khẩu dạng chữ thô (ví dụ: 123456) để dễ kiểm tra. Sang Buổi 5, chúng ta sẽ học cách băm (Hash) mật khẩu này để bảo mật.
3. **Mối liên kết:** Sau khi tạo xong User, hãy mở **SQL Server Management Studio (SSMS)** để kiểm tra xem bảng **Users** đã có dữ liệu chưa. Đây sẽ là tài liệu để chúng ta test chức năng Login ở bài học tiếp theo.

4. GIẢI THÍCH CHO NGƯỜI HỌC

- **Tại sao dùng `type="password"`?** Để đảm bảo tính riêng tư, không cho người đứng cạnh nhìn thấy mật khẩu khi đang nhập.
- **Tại sao cần Role?** Nếu không có Role, hệ thống sẽ coi mọi người dùng là như nhau. Việc phân quyền giúp tách biệt người có quyền "Xóa/Sửa" (Admin) và người chỉ có quyền "Viết bài" (Editor).

Hoàn tất phần này, hệ thống của chúng ta đã sẵn sàng 100% về mặt dữ liệu để triển khai phần **Authentication & Authorization** ở Buổi 5.

Để hoàn thiện thực hành **Quản trị User**, chúng ta cần xử lý hai thao tác còn lại: **Sửa (Edit)** và **Xóa (Delete)**.

Đặc biệt, phần **Sửa** mật khẩu là phần dễ gây lỗi nhất đối với sinh viên. Dưới đây là nội dung chi tiết để in cho sinh viên thực hành.

THAO TÁC SỬA VÀ XÓA THÀNH VIÊN (USER EDIT & DELETE)

Mục tiêu: Giúp sinh viên làm chủ việc cập nhật thông tin thành viên và quản lý vòng đời tài khoản, xử lý tình huống đổi mật khẩu hoặc giữ nguyên mật khẩu cũ.

Đảm bảo chèn các nút lệnh Sửa và Xóa cho mỗi dòng chứa thành viên trong trang quản trị viên, chèn trong file View/User/Index.cshtml đoạn code sau

...

```
<a asp-action="Edit" asp-route-id="@user.Id" class="btn btn-outline-warning">
    Sửa
</a>
<a asp-action="Delete" asp-route-id="@user.Id" class="btn btn-outline-danger"
onclick="return confirm('Bạn có chắc chắn muốn xóa thành viên này?')">
    Xóa
</a>
```

....

1. CHỨC NĂNG SỬA THÀNH VIÊN (USER EDIT)

Chức năng này cần lưu ý: Nếu người quản trị chỉ sửa tên/quyền mà để trống ô mật khẩu, hệ thống phải giữ lại mật khẩu cũ.

A. Tại UserController.cs

Thêm 2 hàm (GET và POST) để xử lý việc hiển thị và cập nhật dữ liệu:

C#

```
// GET: Hiển thị form kèm dữ liệu cũ của User
[HttpGet]
public IActionResult Edit(int id)
{
    var user = _context.Users.Find(id);
    if (user == null) return NotFound();

    return View(user);
}

// POST: Thực hiện lưu thay đổi
[HttpPost]
public IActionResult Edit(User model, string NewPassword)
{
    // 1. Tìm User gốc trong Database để lấy lại mật khẩu cũ nếu cần
    var existingUser = _context.Users.AsNoTracking().FirstOrDefault(u => u.Id == model.Id);

    if (existingUser == null) return NotFound();

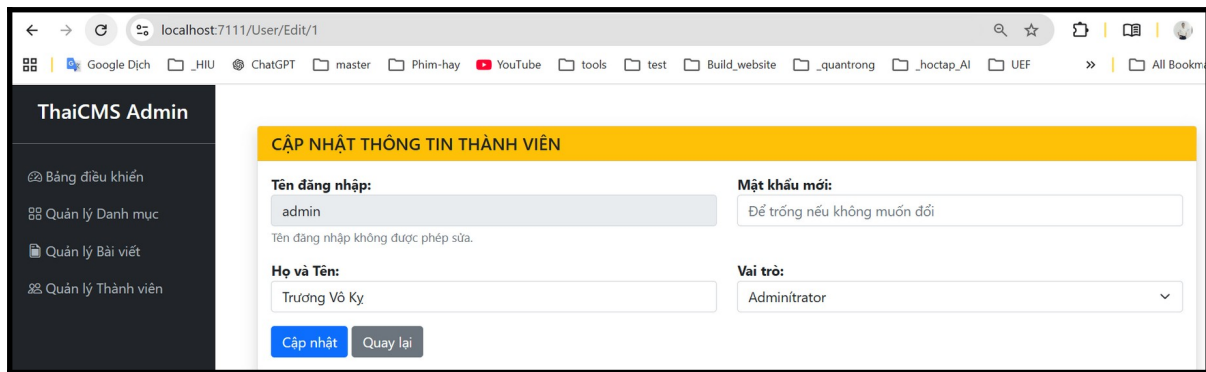
    // 2. Xử lý mật khẩu: Nếu nhập mới thì lấy cái mới, nếu trống thì lấy cái cũ
    if (!string.IsNullOrEmpty(NewPassword))
    {
        model.PasswordHash = NewPassword; // Sau này sẽ mã hóa tại đây
    }
    else
    {
        model.PasswordHash = existingUser.PasswordHash;
    }

    // 3. Cập nhật vào Database
    _context.Users.Update(model);
    _context.SaveChanges();

    return RedirectToAction("Index");
}
```

B. Tại View (Views/User/Edit.cshtml)

Thiết kế Form sửa tương tự trang Create nhưng bổ sung thẻ ẩn `Id`.



Razor CSHTML

```
@model CMS.Data.Entities.User
```

```
@{
    Layout = "_LayoutAdmin";
}
```

```
<div class="container mt-4">
    <div class="card shadow">
        <div class="card-header bg-warning text-dark">
            <h5 class="mb-0">CẬP NHẬT THÔNG TIN THÀNH VIÊN</h5>
        </div>
```

```
        <div class="card-body">
            <form asp-action="Edit" method="post">
                <input type="hidden" asp-for="Id" />
```

```
                <div class="row mb-3">
                    <div class="col-md-6">
                        <label class="fw-bold">Tên đăng nhập:</label>
                        <input asp-for="Username" class="form-control" readonly />
                        <small class="text-muted">Tên đăng nhập không được phép sửa.</small>
                    </div>
```

```
                    <div class="col-md-6">
                        <label class="fw-bold">Mật khẩu mới:</label>
                        <input type="password" name="NewPassword" class="form-control"
placeholder="Để trống nếu không muốn đổi" />
                    </div>
                </div>
```

```
            </div>
```

```
                <div class="row mb-3">
                    <div class="col-md-6">
                        <label class="fw-bold">Họ và Tên:</label>
                        <input asp-for="FullName" class="form-control" required />
                    </div>
                    <div class="col-md-6">
                        <label class="fw-bold">Vai trò:</label>
                        <select asp-for="Role" class="form-select">
                            <option value="Admin">Admin</option>
                            <option value="Editor">Editor</option>
                        </select>
                    </div>
                </div>
```

```
        </select>
    </div>
</div>

    <button type="submit" class="btn btn-primary">Cập nhật</button>
    <a asp-action="Index" class="btn btn-secondary">Quay lại</a>
</form>
</div>
</div>
</div>
```

2. CHỨC NĂNG XÓA THÀNH VIÊN (USER DELETE)

A. Tại UserController.cs

Hàm xử lý xóa trực tiếp từ Id nhận được từ giao diện.

C#

```
public IActionResult Delete(int id)
{
    var user = _context.Users.Find(id);
    if (user != null)
    {
        _context.Users.Remove(user);
        _context.SaveChanges();
    }
    return RedirectToAction("Index");
}
```

B. Tại View (Index.cshtml)

Cập nhật cột Thao tác để gọi lệnh xóa:

Razor CSHTML

```
<a asp-action="Delete" asp-route-id="@user.Id"
    class="btn btn-sm btn-outline-danger"
    onclick="return confirm('Bạn có chắc chắn muốn xóa thành viên @user.Username không?')">
    Xóa
</a>
```

3. GIẢI THÍCH CHO SINH VIÊN

1. **Tại sao Username dùng readonly?**: Trong các hệ thống thực tế, Username thường là định danh duy nhất (Unique) gắn liền với lịch sử bài viết. Sửa Username có thể dẫn đến lỗi toàn vẹn dữ liệu.

2. **Cơ chế AsNoTracking():** Khi muốn so sánh dữ liệu mới gửi về và dữ liệu cũ trong DB mà không làm "rối" Entity Framework, chúng ta dùng lệnh này để đọc dữ liệu một cách độc lập.
 3. **Mật khẩu (PasswordHash):** Nhấn mạnh rằng nếu không xử lý logic `if (string.IsNullOrEmpty(NewPassword))`, mật khẩu của User sẽ bị ghi đè thành rỗng và họ sẽ không bao giờ đăng nhập lại được.
-

4. BÀI TẬP KIỂM CHỨNG (CHECKPOINT)

- **Test 1:** Sửa Họ tên của User "admin" nhưng không nhập mật khẩu mới. Sau khi lưu, hãy dùng SSMS kiểm tra cột PasswordHash xem mật khẩu cũ có còn không.
- **Test 2:** Thử xóa một User mới tạo và kiểm tra xem dòng đó đã biến mất trong SQL Server chưa.
- **Test 3:** Sửa Role của một User từ "Editor" sang "Admin" và kiểm tra xem nhãn (Badge) ở trang Index có đổi từ màu Xanh sang màu Đỏ không.

BUỔI 5: BẢO MẬT & PHÂN QUYỀN (SECURITY & IDENTITY)

Thời lượng: 5 tiết

Mục tiêu bài học:

- Hiểu và triển khai cơ chế **Authentication** (Xác thực người dùng).
- Hiểu và triển khai cơ chế **Authorization** (Phân quyền quản trị).
- Nắm vững kỹ thuật **Mã hóa mật khẩu** (Password Hashing) để bảo vệ dữ liệu thành viên.

1. KHÁI NIỆM TRỌNG TÂM: LOGIN FLOW

Trước khi viết code, **chúng ta** hiểu luồng hoạt động của một hệ thống có bảo mật:

- Người dùng nhập Username/Password.
- Hệ thống kiểm tra trong Database (Bảng Users).
- Nếu khớp, hệ thống cấp một "chiếc thẻ bài" (Cookie hoặc Token) để người dùng có thể truy cập vào trang Admin.

2. THỰC HÀNH

Phần A: Xây dựng trang đăng nhập (Login)

Mục tiêu:

- Triển khai cơ chế xác thực người dùng (**Authentication**) bằng Cookie.
- Thiết lập phân quyền (**Authorization**) theo vai trò: Admin và Editor.
- Thực hiện chức năng Đăng nhập và Đăng xuất cho hệ thống ThaiCMS.

BƯỚC 1: CẤU HÌNH DỊCH VỤ XÁC THỰC (PROGRAM.CS)

Trước khi viết code chức năng, cần khai báo với hệ thống về việc sử dụng xác thực bằng Cookie.

Thao tác: Mở file `Program.cs`, tìm đoạn trước dòng `var app = builder.Build();` và thêm đoạn mã sau:

C#

```
// 1. Khai báo dịch vụ xác thực Cookie
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
```

```
{
    options.LoginPath = "/Account/Login"; // Đường dẫn nếu chưa đăng nhập
    options.AccessDeniedPath = "/Account/AccessDenied"; // Đường dẫn nếu vào trang không
    được phép
});
```

Lưu ý: Trước dòng `app.UseAuthorization();` (phía dưới), phải đảm bảo đã có dòng `app.UseAuthentication();`.

Nghĩa là

`app.UseAuthentication();` // BƯỚC A: Xác nhận "Anh là ai?" (Kiểm tra thẻ bài)
`app.UseAuthorization();` // BƯỚC B: Xác nhận "Anh được làm gì?" (Kiểm tra quyền)

BƯỚC 2: TẠO ACCOUNTCONTROLLER

Controller này đóng vai trò điều hướng các thao tác liên quan đến tài khoản.

Thao tác: Chuột phải vào thư mục `Controllers` -> Add -> Controller -> MVC Controller - Empty -> Đặt tên `AccountController.cs`.

Mã nguồn ban đầu:

C#

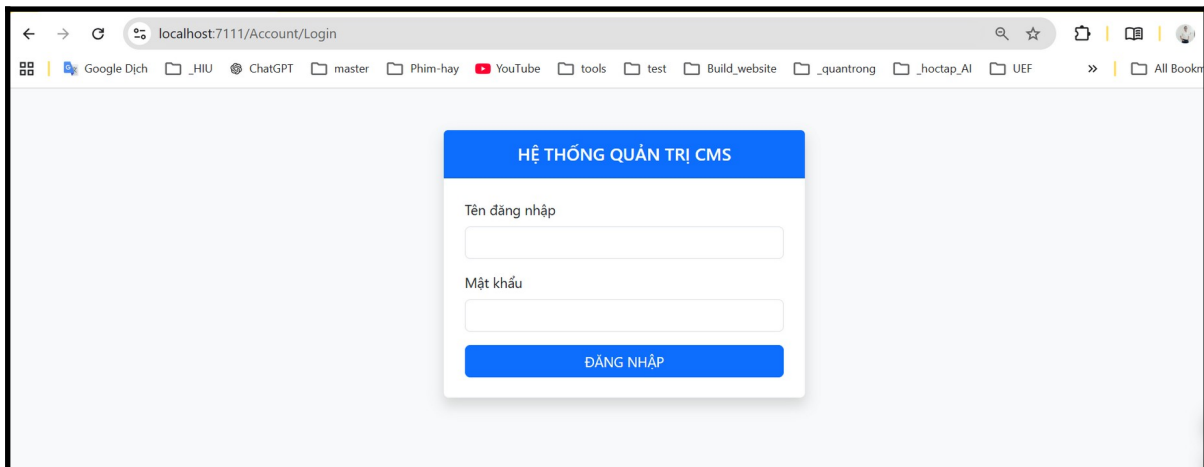
```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using System.Security.Claims;
using CMS.Data; // Thay bằng Namespace của project Data
```

```
public class AccountController : Controller
{
    private readonly ApplicationDbContext _context;

    public AccountController(ApplicationDbContext context)
    {
        _context = context;
    }

    [HttpGet]
    public IActionResult Login()
    {
        return View();
    }
}
```

BƯỚC 3: THIẾT KẾ GIAO DIỆN ĐĂNG NHẬP (LOGIN.CSHTML)



Thao tác: Chuột phải vào thư mục Views -> Tạo thư mục mới tên Account -> Tạo file Login.cshtml.

Mã nguồn giao diện:

Razor CSHTML

```
@{
    Layout = null; // Trang đăng nhập độc lập, không dùng Layout chung
}
<!DOCTYPE html>
<html>
<head>
    <title>Đăng nhập hệ thống</title>
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
</head>
<body class="bg-light">
    <div class="container mt-5">
        <div class="row justify-content-center">
            <div class="col-md-4">
                <div class="card shadow border-0">
                    <div class="card-header bg-primary text-white text-center py-3">
                        <h5 class="mb-0">HỆ THỐNG QUẢN TRỊ CMS</h5>
                    </div>
                    <div class="card-body p-4">
                        <form asp-action="Login" method="post">
                            <div class="mb-3">
                                <label class="form-label">Tên đăng nhập</label>
                                <input type="text" name="username" class="form-control" required />
                            </div>
                            <div class="mb-3">
                                <label class="form-label">Mật khẩu</label>
                                <input type="password" name="password" class="form-control" required />
                            </div>
                        </form>
                    </div>
                </div>
            </div>
        </div>
    </div>
</body>
</html>
```



```
return View();  
}  
  
// Hàm đăng xuất  
public async Task<ActionResult> Logout()  
{  
    await HttpContext.SignOutAsync(CookieAuthenticationDefaults.AuthenticationScheme);  
    return RedirectToAction("Login");  
}
```

BƯỚC 5: GIẢI THÍCH CÁC KHÁI NIỆM CƠ BẢN

Để hiểu rõ luồng bảo mật của ASP.NET Core, cần nắm vững 3 thành phần:

1. **Claim:** Là một mẫu thông tin về người dùng (ví dụ: Tên, Email, Quyền hạn).
2. **Identity:** Là "chứng minh nhân dân" chứa các mẫu thông tin (Claims) trên.
3. **Principal:** Là "người sở hữu" chứng minh nhân dân đó. Khi truy cập trang web, hệ thống kiểm tra Principal để cho phép hoặc từ chối vào cửa.

BƯỚC 6: KIỂM CHỨNG KẾT QUẢ (CHECKPOINT)

1. **Đăng nhập thành công:** Nhập đúng tài khoản trong bảng `Users`, hệ thống phải chuyển hướng vào trang quản trị.
2. **Kiểm tra Cookie:** Nhấn **F12** trên trình duyệt -> Chọn tab **Application** -> **Cookies**. Phải thấy xuất hiện dòng `.AspNetCore.Cookies`.
3. **Thử nghiệm Logout:** Nhấn vào link Đăng xuất, hệ thống phải xóa Cookie và quay về trang Login.

Phần B: BẢO VỆ CÁC TRANG QUẢN TRỊ (AUTHORIZATION)

Mục tiêu bài học:

- Sử dụng thuộc tính `[Authorize]` để khóa các khu vực nhạy cảm.
- Thực hiện phân quyền chi tiết theo vai trò (**Role-based Authorization**).
- Xử lý lỗi khi người dùng truy cập vào trang không có quyền (**Access Denied**).

BƯỚC 1: KHÓA TRANG ADMIN (AUTHENTICATION CHECK)

Sau khi đã có chức năng đăng nhập ở Tiết 1 & 2, chúng ta cần chặn tất cả những người chưa đăng nhập không được vào xem dữ liệu.

Thao tác: Mở các Controller sau và thêm thuộc tính `[Authorize]` lên ngay phía trên tên Class:

- `CategoryController.cs`
- `PostController.cs`
- `UserController.cs`

Mã nguồn ví dụ:

C#

```
using Microsoft.AspNetCore.Authorization; // Cần thêm namespace này

[Authorize] // Bắt buộc phải đăng nhập mới được vào các hàm bên dưới
public class PostController : Controller
{
    // ... các code hiện tại ...
}
```

Giải thích: Khi có thuộc tính này, nếu một người dùng cố tình gõ trực tiếp URL `/Post/Create` mà chưa đăng nhập, hệ thống sẽ tự động "đá" họ về trang `/Account/Login` như đã cấu hình trong `Program.cs`.

BƯỚC 2: PHÂN QUYỀN THEO VAI TRÒ (ROLE-BASED)

Trong hệ thống CMS, không phải ai cũng có quyền quản lý thành viên (User). Chúng ta sẽ chỉ cho phép người có Role là "Admin" mới được vào `UserController`.

Thao tác: Mở file `UserController.cs` và cập nhật thuộc tính `[Authorize]` như sau:

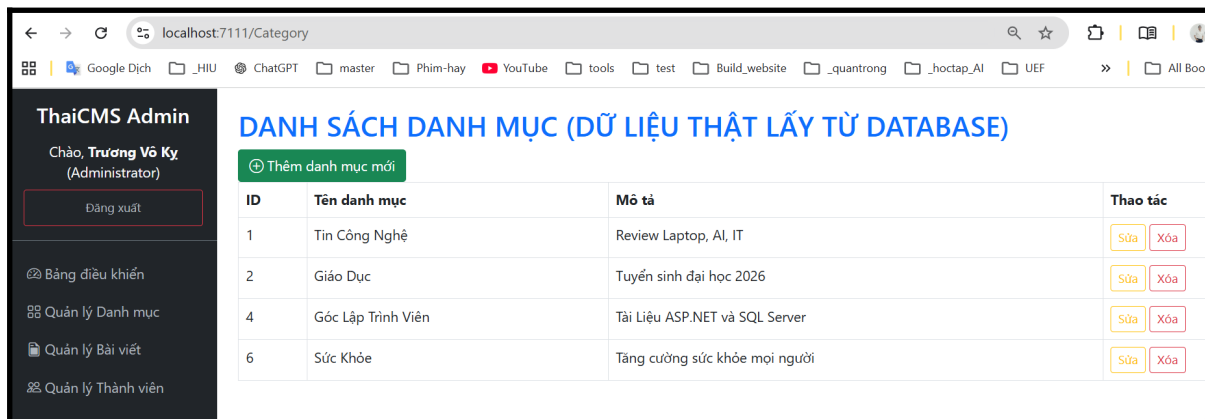
C#

```
[Authorize(Roles = "Admin")] // Chỉ tài khoản có Role là Admin mới được phép vào
public class UserController : Controller
{
    // ... các code hiện tại ...
}
```

Giải thích: Nếu một người đăng nhập bằng tài khoản có Role là "Editor", họ vẫn xem được Bài viết nhưng khi bấm vào Quản lý thành viên, hệ thống sẽ chặn lại vì không đúng quyền hạn.

BƯỚC 3: HIỂN THỊ THÔNG TIN NGƯỜI DÙNG TRÊN LAYOUT

Để giao diện chuyên nghiệp, chúng ta sẽ hiển thị tên người đang đăng nhập và nút Đăng xuất trên thanh điều hướng.



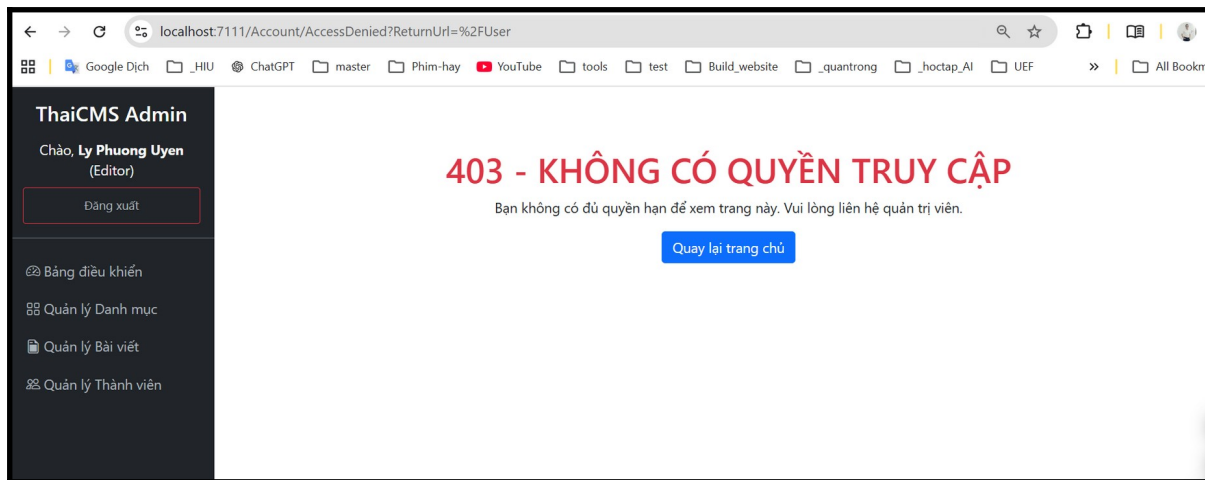
Thao tác: Mở file `_LayoutAdmin.cshtml`, tìm vị trí Header hoặc Sidebar và thêm đoạn mã sau:

Razor CSHTML

```
<ul class="navbar-nav ms-auto">
    @if (User.Identity.IsAuthenticated)
    {
        <li class="nav-item">
            <span class="nav-link text-dark">
                Chào, <strong>@User.FindFirst("FullName")?.Value</strong>
                (@User.FindFirst(System.Security.Claims.ClaimTypes.Role)?.Value)
            </span>
        </li>
        <li class="nav-item">
            <a class="btn btn-outline-danger btn-sm" asp-controller="Account" asp-
            action="Logout">Đăng xuất</a>
        </li>
    }
</ul>
```

BƯỚC 4: XỬ LÝ TRANG TỪ CHỐI TRUY CẬP (ACCESS DENIED)

Khi một Editor cố tình vào trang của Admin, chúng ta cần hiển thị một thông báo lịch sự thay vì để trình duyệt báo lỗi hệ thống.



Thao tác: Thêm hàm sau vào `AccountController.cs`:

C#

```
[HttpGet]
public IActionResult AccessDenied()
{
    return View();
}
```

Tạo View `AccessDenied.cshtml` trong thư mục `Views/Account`:

Razor CSHTML

```
@{
    Layout = "_LayoutAdmin";
}
<div class="container mt-5 text-center">
    <h1 class="text-danger">403 - KHÔNG CÓ QUYỀN TRUY CẬP</h1>
    <p>Bạn không có đủ quyền hạn để xem trang này. Vui lòng liên hệ quản trị
    viên.</p>
    <a href="/" class="btn btn-primary">Quay lại trang chủ</a>
</div>
```

BƯỚC 5: KIỂM CHỨNG KẾT QUẢ (CHECKPOINT)

1. **Chưa đăng nhập:** Mở trình duyệt ẩn danh, gõ `/Category`. Hệ thống phải chuyển về trang Login.
2. **Đăng nhập quyền Editor:** Vào mục Quản lý bài viết (OK), vào mục Quản lý thành viên -> Hệ thống phải chuyển về trang Access Denied.
3. **Đăng nhập quyền Admin:** Phải vào được tất cả các trang mà không bị chặn.

Lưu ý:

Trong bài tập này, chúng ta lưu mật khẩu hiện (Plain Text) để dễ học và dễ sửa lỗi. Khi đi làm thực tế, các em sẽ cần thêm một bước mã hóa để bảo mật hơn, bước đó chúng ta sẽ tìm hiểu sau khi đã thành thạo luồng này

BUỔI 6: WEB API & RESTFUL SERVICE

Mục tiêu bài học:

- Hiểu kiến trúc **Client - Server** và vai trò của Web API.
- Làm quen với định dạng dữ liệu **JSON**.
- Triển khai các chuẩn **RESTful**: GET, POST, PUT, DELETE.
- Tạo "cửa ngõ" dữ liệu để Buổi 7 kết nối với giao diện **ReactJS**.

1. KHÁI NIỆM TRỌNG TÂM: API LÀ GÌ?

Trong các buổi trước, chúng ta dùng **MVC** (Model-View-Controller) để trả về các trang web (HTML). Tuy nhiên, để làm việc với các ứng dụng hiện đại như ReactJS, Mobile App, Backend không trả về giao diện mà chỉ trả về **Dữ liệu thô (JSON)**.

- **API (Application Programming Interface)**: Là cầu nối cho phép hai ứng dụng "nói chuyện" với nhau.
- **JSON (JavaScript Object Notation)**: Là ngôn ngữ chung để truyền tải dữ liệu, có cấu trúc dạng `{ "key": "value" }`.

1. Sự khác biệt giữa Web truyền thống (MVC) và Web hiện đại (API + Frontend)

Cách làm cũ (ASP.NET Core MVC):

- **Cơ chế**: Khi người dùng yêu cầu một trang web, Server sẽ thực hiện: Lấy dữ liệu -> Gắn dữ liệu vào file HTML (View) -> Trả về nguyên một trang web hoàn chỉnh.
- **Nhược điểm**: Server vừa phải lo dữ liệu, vừa phải lo giao diện. Nếu muốn làm thêm ứng dụng Mobile, bạn phải viết lại toàn bộ từ đầu vì Mobile không đọc được file HTML của Web.

Cách làm hiện đại (Web API):

- **Cơ chế**: Server chỉ tập trung làm đúng một việc: **Cung cấp dữ liệu thô**.
- **Giao diện**: ReactJS (Web), Flutter (Mobile) hay Swift (iOS) sẽ tự xây dựng giao diện riêng của mình. Khi cần dữ liệu, chúng "gõ cửa" Server để lấy.
- **Ưu điểm**: Một Backend (API) có thể phục vụ cùng lúc cho nhiều nền tảng (Web, App, IoT...).

2. JSON là gì? (Ngôn ngữ chung của thế giới số)

Nếu coi API là một người giao hàng, thì **JSON** chính là cách đóng gói kiện hàng đó. Trước đây người ta dùng XML (rất rườm rà), nay JSON chiếm ưu thế vì nó cực kỳ nhẹ và dễ đọc.

Cấu trúc đặc trưng:

- Dữ liệu luôn đi theo cặp **Key** (Tên trường) và **Value** (Giá trị).
- Ví dụ một bài viết khi trả về từ API sẽ trông như thế này:

```
JSON
{
  "id": 101,
  "title": "Học lập trình Web Buổi 6",
  "author": "Thái GV",
  "isPublished": true,
  "tags": ["API", "Backend", "JSON"]
}
```

- **Tại sao sinh viên cần biết cái này?** Vì khi làm việc với ReactJS, các em sẽ phải viết code để "bóc tách" từng cái **key** này (như `post.title`, `post.author`) để hiển thị lên màn hình.
-

3. API - "Thực đơn" của Server dành cho Client

Để sinh viên dễ hiểu về API (Application Programming Interface), hãy dùng hình ảnh ẩn dụ về **Nhà hàng**:

1. **Khách hàng (Frontend/ReactJS):** Ngồi ở bàn và muốn ăn món "Danh sách bài viết".
 2. **Thực đơn (API Documentation):** Liệt kê các món mà nhà hàng có thể phục vụ (Ví dụ: `GET /api/posts`).
 3. **Người bồi bàn (API):** Tiếp nhận yêu cầu từ khách, chuyển xuống bếp, và mang món ăn (Dữ liệu JSON) lên cho khách. Khách muốn ăn bằng bát sứ hay đĩa nhựa (Giao diện React hay Mobile) là quyền của khách, bồi bàn không can thiệp.
-

4. Tại sao Backend chỉ trả về dữ liệu thô?

- **Tiết kiệm băng thông:** Thay vì gửi cả một trang HTML nặng 1MB (có cả CSS, JS, giao diện), Server chỉ gửi 1 đoạn chữ JSON nặng 1KB. Website sẽ load cực nhanh.
 - **Phân tách trách nhiệm (Decoupling):** Đội làm Backend chỉ cần giỏi SQL, C#. Đội làm Frontend chỉ cần giỏi ReactJS, CSS. Hai đội không cần can thiệp vào code của nhau, chỉ cần thống nhất với nhau qua cái "thực đơn" API.
-

2. NỘI DUNG THỰC HÀNH

Phần 1: KHỞI TẠO API CONTROLLER

Mục tiêu: Tạo ra một Controller không có giao diện, chỉ làm nhiệm vụ nhận yêu cầu và trả về dữ liệu JSON cho Frontend (ReactJS).

BƯỚC 1: THAO TÁC TẠO FILE TRÊN VISUAL STUDIO

Sinh viên cần thực hiện chính xác trình tự chuột phải để Visual Studio tự động sinh ra các thư viện cần thiết cho API.

1. Tại cửa sổ **Solution Explorer**, tìm đến thư mục **Controllers**.
2. **Chuột phải** vào thư mục **Controllers** -> Chọn **Add** -> Chọn **Controller...**
3. Một bảng hiện ra, hãy nhìn menu bên trái và chọn mục **API**.
4. Tại cột giữa, chọn **API Controller - Empty**.
 - o *Lưu ý quan trọng:* Tuyệt đối không chọn "MVC Controller" vì nó sẽ chứa các hàm trả về View (giao diện), không phù hợp để làm API.
5. Nhấn nút **Add**.
6. Tại ô **Controller name**, đặt tên là: `PostsController.cs` (có chữ **s** số nhiều để phân biệt với bài viết đơn lẻ). Nhấn **Add** để hoàn tất.

BƯỚC 2: VIẾT MÃ NGUỒN CƠ BẢN

Mở file `PostsController.cs` vừa tạo và cập nhật nội dung code như sau. Hãy chú ý các dòng chú thích để hiểu ý nghĩa:

```
C#
```

```
using Microsoft.AspNetCore.Mvc;  
using CMS.Data; // Thay bằng Namespace của project chứa ApplicationDbContext của bạn
```

```
namespace CMS.Backend.Controllers
```

```
{
```

```
    // 1. Định nghĩa đường dẫn để gọi API. [controller] sẽ tự lấy tên "Posts"  
    // Khi chạy, địa chỉ sẽ là: https://localhost:xxxx/api/posts  
    [Route("api/[controller]")]
```

```
    // 2. Đánh dấu đây là một API Controller để hệ thống hỗ trợ các tính năng RESTful  
    [ApiController]
```

```
    // 3. API Controller phải kế thừa từ ControllerBase (thay vì Controller như MVC)
```

```
    public class PostsController : ControllerBase
```

```
    {
```

```
        // 4. Khai báo biến kết nối Database  
        private readonly ApplicationDbContext _context;
```

```
        // 5. Hàm khởi tạo (Constructor): "Tiêm" kết nối Database vào để sử dụng  
        public PostsController(ApplicationDbContext context)
```

```
        {  
            _context = context;  
        }  
    }
```

```
}
```

BƯỚC 3: GIẢI THÍCH CHI TIẾT CÁC THÀNH PHẦN

Để sinh viên không chỉ copy-code mà còn hiểu bài, cần giải thích 3 điểm khác biệt sau:

1. **[Route("api/[controller]")]**: Trong MVC truyền thống, đường dẫn thường là `/[Controller]/[Action]`. Trong API, chúng ta luôn thêm tiền tố **api/** phía trước để phân biệt rõ ràng: Đây là đường dẫn lấy dữ liệu, không phải đường dẫn xem trang web.
 2. **[ApiController]**: Thuộc tính này cực kỳ quyền năng. Nó tự động kiểm tra dữ liệu đầu vào (Validation), tự động trả về lỗi 400 nếu dữ liệu sai, giúp chúng ta bớt phải viết code kiểm tra thủ công.
 3. **ControllerBase**: Trong các buổi trước (MVC), chúng ta dùng **Controller**. Nhưng ở API, chúng ta dùng **ControllerBase**. Lý do là API không cần các tính năng hiển thị giao diện (View), sử dụng **ControllerBase** sẽ giúp ứng dụng nhẹ hơn và tối ưu hơn.
-

BƯỚC 4: KIỂM CHỨNG (CHECKPOINT)

Sau khi viết xong đoạn code trên, hãy yêu cầu sinh viên:

1. Nhấn **Build** (Ctrl + Shift + B) để đảm bảo không có lỗi cú pháp.
 2. **Giải thích câu hỏi**: "Tại sao hiện tại truy cập vào đường dẫn `api/posts` vẫn chưa thấy gì?".
 - *Đáp án*: Vì chúng ta mới chỉ xây dựng "cái khung" (Class), chưa viết các "hành động" (Action) như lấy danh sách hay chi tiết bài viết. Các hành động này sẽ được thực hiện ở Tiết 2.
-

Phần 2: API LẤY DANH SÁCH BÀI VIẾT (GET METHOD)

Mục tiêu: Xây dựng các hàm để lấy dữ liệu bài viết từ Database và trả về dưới dạng JSON. Đây chính là "nguồn thức ăn" để giao diện ReactJS có thể hiển thị danh sách bài viết lên trang chủ.

BƯỚC 1: VIẾT HÀM LẤY TOÀN BỘ BÀI VIẾT

Chúng ta sẽ viết hàm `GetAll` để lấy các bài viết mới nhất. Hãy thêm đoạn code này vào bên trong Class `PostsController`.

Mã nguồn chi tiết:

C#

```
// 1. Chỉ định đây là phương thức GET (Dùng để lấy dữ liệu)
[HttpGet]
public IActionResult GetAll()
{
    // Lấy dữ liệu từ bảng Posts
    var posts = _context.Posts
        .OrderByDescending(p => p.Id) // Sắp xếp bài mới nhất lên đầu
        .Select(p => new {           // "Gọt tỉa" dữ liệu: chỉ lấy những trường cần thiết
            p.Id,
            p.Title,
            p.ImageUrl,
            p.CreatedAt,
            CategoryName = p.Category.Name // Lấy tên danh mục thay vì chỉ lấy ID
        })
        .ToList();

    // Trả về kết quả cho Frontend kèm mã trạng thái 200 (Thành công)
    return Ok(posts);
}
```

Giải thích:

- **[HttpGet]**: Khi ReactJS gọi đến địa chỉ `api/posts` bằng phương thức GET, hàm này sẽ tự động chạy.
- **.Select(...)**: Trong Database có rất nhiều thông tin (như Password người đăng, nội dung chi tiết rất dài...), nhưng trang danh sách chỉ cần Tiêu đề và Ảnh. Việc `Select` giúp gọt tin JSON nhẹ hơn, giúp web load nhanh hơn.
- **return Ok(posts)**: Lệnh này sẽ tự động biến danh sách C# thành định dạng JSON.

BƯỚC 2: VIẾT HÀM LẤY BÀI VIẾT THEO DANH MỤC

Khi người dùng nhấn vào một danh mục (ví dụ: "Tin tức"), chúng ta chỉ muốn hiển thị các bài thuộc danh mục đó.

Mã nguồn chi tiết:

C#

```
// 2. Định nghĩa đường dẫn có tham số: api/posts/category/{id}
[HttpGet("category/{categoryId}")]
public IActionResult GetByCategory(int categoryId)
{
    // Lọc các bài viết có CategoryId trùng với ID truyền vào từ URL
    var posts = _context.Posts
        .Where(p => p.CategoryId == categoryId)
        .Select(p => new {
            p.Id,
            p.Title,
            p.ImageUrl,
            p.CreatedAt
        })
        .ToList();

    return Ok(posts);
}
```

Giải thích cho sinh viên:

- **{categoryId}**: Đây là một biến động trên URL. Ví dụ nếu gõ `api/posts/category/1`, hệ thống sẽ hiểu `categoryId = 1`.
- **.Where(...)**: Đây là câu lệnh lọc dữ liệu (tương đương lệnh `WHERE` trong SQL).

BƯỚC 3: THAO TÁC KIỂM TRA (TESTING)

Vì chưa có giao diện ReactJS, chúng ta sẽ dùng trình duyệt hoặc Swagger để kiểm tra xem "bàn" API có làm việc đúng không.

1. **Nhấn F5** để chạy dự án Backend.
2. Nếu hiện ra trang **Swagger**, tìm đến mục `GET /api/Posts`, nhấn **Try it out -> Execute**.
3. **Quan sát kết quả (Response body)**: Bạn sẽ thấy dữ liệu hiện ra trong dấu ngoặc vuông [...], bên trong là các đối tượng nằm trong dấu {...}. Đó chính là JSON.

 LƯU Ý

Checkpoint: Nếu sinh viên thấy được danh sách bài viết hiện ra trong trình duyệt dưới dạng chữ thô (JSON), nghĩa là các em đã thành công 50% chặng đường của Buổi 6!

Phần 3: API CHI TIẾT BÀI VIẾT (GET BY ID)

Mục tiêu: Xây dựng hàm lấy đầy đủ thông tin của **một bài viết cụ thể** dựa trên ID. Đây là dữ liệu quan trọng để ReactJS hiển thị nội dung bài báo khi người dùng nhấn vào tiêu đề hoặc ảnh ở trang chủ.

BƯỚC 1: VIẾT HÀM LẤY CHI TIẾT BÀI VIẾT

Chúng ta sẽ sử dụng tham số ngay trên đường dẫn URL (ví dụ: `api/posts/5`) để xác định bài viết cần lấy.

Thao tác: Thêm đoạn mã này vào sau hàm `GetByCategory` trong file `PostsController.cs`.

C#

```
// 1. Định nghĩa đường dẫn nhận ID: api/posts/{id}
[HttpGet("{id}")]
public IActionResult GetDetail(int id)
{
    // 2. Tìm bài viết đầu tiên có Id khớp với tham số truyền vào
    var post = _context.Posts
        .FirstOrDefault(p => p.Id == id);

    // 3. Xử lý trường hợp không tìm thấy (ID không tồn tại)
    if (post == null)
    {
        // Trả về lỗi 404 kèm thông báo dưới dạng JSON
        return NotFound(new { message = "Không tìm thấy bài viết này trong hệ thống" });
    }

    // 4. Trả về bài viết tìm thấy kèm mã 200 (Thành công)
    return Ok(post);
}
```

BƯỚC 2: GIẢI THÍCH CHI TIẾT

- Tham số {id}:** Đây là placeholder. Khi chạy thực tế, {id} sẽ được thay bằng số cụ thể.
 - Ví dụ: Gọi `api/posts/10` -> hệ thống tự gán `id = 10`.
- FirstOrDefault:** Lệnh này sẽ quét trong bảng `Posts`. Nếu tìm thấy, nó trả về đối tượng bài viết; nếu không thấy, nó trả về `null`.
- Tại sao không dùng .Select() như Tiết 2?:**
 - Ở trang danh sách (Tiết 2), chúng ta chỉ lấy một vài trường cho nhẹ.
 - Ở trang chi tiết này, chúng ta cần **toàn bộ nội dung** (bao gồm cả trường `Content` chứa mã HTML từ trình soạn thảo) nên chúng ta trả về nguyên đối tượng `post`.

4. **Thông báo lỗi NotFound:** Trong API, thay vì chuyển hướng người dùng sang trang báo lỗi 404 rườm rà, chúng ta chỉ cần trả về mã lỗi 404 kèm một "gói tin" JSON nhỏ để Frontend tự quyết định cách hiển thị thông báo cho người dùng.

BƯỚC 3: THAO TÁC KIỂM TRA (CHECKPOINT)

Sinh viên thực hiện kiểm tra theo 2 kịch bản:

- **Kịch bản 1 (Thành công):** 1. Chạy dự án.
2. Gõ lên trình duyệt: `https://localhost:xxxx/api/posts/1` (Thay 1 bằng một ID đang có trong DB).
3. Kết quả: Phải hiện ra dữ liệu của duy nhất 1 bài viết đó.
- **Kịch bản 2 (Thất bại):**
 1. Gõ lên trình duyệt một ID chắc chắn không có (ví dụ: 9999).
 2. Kết quả: Trình duyệt báo lỗi 404 hoặc hiện dòng chữ JSON: `{"message": "Không tìm thấy bài viết này trong hệ thống"}`.

MẸO

Lưu ý về thứ tự code: Dặn sinh viên các hàm có `[HttpGet]` phải đặt tên Route khác nhau hoặc có tham số khác nhau để tránh bị lỗi "AmbiguousMatchException" (Hệ thống không biết gọi hàm nào).

- **Kiểm tra dữ liệu:** Nhắc sinh viên nếu bài viết có nội dung là HTML (từ CKEditor), dữ liệu trả về sẽ có các thẻ `<p>`, `<strong>`... Điều này là hoàn toàn chính xác vì ReactJS sẽ cần những thẻ này để hiển thị đúng định dạng.

Phần 4: KIỂM TRA API VỚI SWAGGER (TESTING API)

Mục tiêu: Vì Web API không có giao diện (HTML) để chúng ta nhấn nút như trang Web thông thường, chúng ta cần sử dụng **Swagger** – một công cụ được tích hợp sẵn trong .NET 8 để "nhìn" thấy các API và chạy thử chúng.

BƯỚC 1: KHỞI ĐỘNG CÔNG CỤ SWAGGER

Mặc định, khi chúng ta tạo dự án Web API trong .NET 8, Microsoft đã cài đặt sẵn thư viện **Swashbuckle** để tự động tạo trang tài liệu API.

Thao tác:

1. Tại Visual Studio, nhấn phím **F5** (hoặc nút Play màu xanh) để chạy dự án.

2. Trình duyệt sẽ tự động mở ra. Nếu đường dẫn chưa đúng, bạn hãy thêm `/swagger` vào sau địa chỉ web (Ví dụ: `https://localhost:7xxx/swagger/index.html`).
3. Bạn sẽ thấy danh sách các Controller hiện ra, trong đó có **Posts**.

BƯỚC 2: THAO TÁC CHẠY THỬ (TRY IT OUT)

Chúng ta sẽ thử nghiệm lấy danh sách bài viết để xem dữ liệu có trả về đúng không.

Thao tác:

1. Nhấn vào dòng **GET /api/Posts** (Hàm lấy toàn bộ bài viết).
2. Nhấn vào nút **Try it out** ở góc bên phải.
3. Nhấn nút **Execute** (màu xanh to) để thực hiện lệnh.

BƯỚC 3: PHÂN TÍCH KẾT QUẢ TRẢ VỀ (RESPONSE)

Sau khi nhấn Execute, bạn cuộn xuống phần **Responses**, đây là nơi quan trọng nhất để sinh viên quan sát:

1. **Code (Mã trạng thái):** Nếu hiện số **200**, nghĩa là API hoạt động hoàn hảo.
2. **Response body (Dữ liệu):** Đây là dữ liệu JSON thực tế từ Database.
 - Kiểm tra xem có đúng định dạng dấu ngoặc vuông [...] cho danh sách không.
 - Các trường dữ liệu (`id`, `title`, `imageUrl`...) có đúng như chúng ta đã lập trình không.
3. **Request URL:** Đây là địa chỉ mà sau này chúng ta sẽ copy để dán vào code bên **ReactJS**.

BƯỚC 4: GIẢI THÍCH CHO SINH VIÊN

Giảng viên cần giải thích ý nghĩa của việc này đối với quy trình làm việc thực tế:

- **Tài liệu sống:** Swagger không chỉ để test, nó còn là "bản cam kết" giữa người làm Backend và người làm Frontend. Nhìn vào đây, người làm ReactJS biết chính xác phải gọi địa chỉ nào và nhận về dữ liệu gì.
- **Không cần giao diện vẫn làm được Backend:** Sinh viên có thể hoàn thiện toàn bộ tính năng thêm, xóa, sửa bài viết thông qua Swagger trước khi bắt tay vào làm giao diện ReactJS.

MẸO "CÀM TAY CHỈ VIỆC" CHO SINH VIÊN

- **Lỗi không hiện Swagger:** Nếu chạy F5 mà báo lỗi trang trắng, hãy nhắc sinh viên kiểm tra file `Program.cs` xem đã có dòng `app.UseSwagger();` và `app.UseSwaggerUI();` chưa (thường nằm trong khối `if (app.Environment.IsDevelopment())`).
- **Sử dụng Postman (Mở rộng):** Nếu sinh viên muốn chuyên nghiệp hơn, giảng viên có thể hướng dẫn tải **Postman**. Thao tác tương tự: Dán URL vào -> Chọn phương thức GET -> Nhấn Send. Postman giúp lưu lại các lượt test cũ rất tiện lợi.

Checkpoint: Nếu sinh viên nhấn **Execute** và thấy danh sách bài viết hiện ra trong ô **Response body**, nghĩa là API của các em đã sẵn sàng để "xuất xưởng" cho phía Frontend sử dụng!

Phần 5: CẤU HÌNH CORS (BẬT ĐÈN XANH CHO REACTJS)

Mục tiêu: Cho phép ứng dụng Frontend (ReactJS) có thể truy cập và lấy dữ liệu từ Backend (Web API). Nếu không có bước này, trình duyệt sẽ chặn mọi dữ liệu vì lý do bảo mật, khiến website của chúng ta không thể hiển thị nội dung.

BƯỚC 1: TẠI SAO PHẢI CẤU HÌNH CORS?

Giải thích cho sinh viên:

- **CORS (Cross-Origin Resource Sharing):** Là một cơ chế bảo mật của trình duyệt.
- **Tình huống:** Web API của chúng ta đang chạy ở một "nhà" (ví dụ: `localhost:7000`), còn ReactJS chạy ở một "nhà" khác (`localhost:3000`).
- **Vấn đề:** Theo mặc định, trình duyệt không cho phép "nhà này" sang "nhà kia" lấy đồ. Lỗi này thường hiện màu đỏ trong cửa sổ Inspect (F12) của trình duyệt với dòng chữ: `"...blocked by CORS policy"`.
- **Giải pháp:** Chúng ta phải cấu hình Backend để xác nhận rằng: "Tôi cho phép ReactJS được quyền lấy dữ liệu của tôi".

BƯỚC 2: THAO TÁC CẤU HÌNH TRONG FILE PROGRAM.CS

Sinh viên thực hiện theo 2 giai đoạn: Đăng ký dịch vụ và Kích hoạt dịch vụ.

Giai đoạn 1: Đăng ký chính sách (Policy)

Thao tác: Mở file `Program.cs`, tìm vị trí **trước** dòng `var app = builder.Build();` và thêm đoạn mã sau:

C#

```
// 1. Khai báo chính sách CORS
builder.Services.AddCors(options => {
    options.AddPolicy("AllowAll", policy => {
        // Cho phép mọi nguồn (Origin), mọi phương thức (GET, POST...), mọi tiêu đề (Header)
        policy.AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader();
    });
});
```

Giai đoạn 2: Kích hoạt chính sách (Middleware)

Thao tác: Vẫn trong file `Program.cs`, tìm vị trí **sau** dòng `app.UseRouting();` và thêm dòng này:

```
C#
```

```
// 2. Kích hoạt chính sách CORS đã khai báo ở trên
app.UseCors("AllowAll");
```

BƯỚC 3: GIẢI THÍCH CHI TIẾT CODE CHO SINH VIÊN

- **AllowAnyOrigin()**: Chấp nhận yêu cầu từ bất kỳ địa chỉ nào (rất tiện khi đang học và thực hành).
- **AllowAnyMethod()**: Chấp nhận tất cả các kiểu gọi như lấy dữ liệu (GET), thêm mới (POST), sửa (PUT), xóa (DELETE).
- **AllowAnyHeader()**: Chấp nhận mọi thông tin đi kèm trong gói tin yêu cầu.
- **Tên chính sách "AllowAll"**: Đây là cái tên chúng ta tự đặt, bạn có thể đặt tên khác nhưng phải đảm bảo ở Bước 1 và Bước 2 phải viết giống hệt nhau.

BƯỚC 4: LƯU Ý QUAN TRỌNG (CẦM TAY CHỈ VIỆC)

1. **Vị trí đặt code:** Nhắc sinh viên phải đặt `app.UseCors("AllowAll");` nằm **giữa** `app.UseRouting();` và `app.UseAuthorization();`. Nếu đặt sai thứ tự, CORS có thể không hoạt động.
2. **Môi trường thực tế:** Giảng viên cần lưu ý sinh viên rằng cách cấu hình `AllowAnyOrigin` chỉ dùng để học tập. Khi đi làm thực tế, chúng ta chỉ nên điền đúng địa chỉ của trang web Frontend vào (ví dụ: `.WithOrigins("https://mywebsite.com")`) để đảm bảo an toàn, tránh bị hacker đánh cắp dữ liệu từ các trang web lạ.

BƯỚC 5: KIỂM CHỨNG (CHECKPOINT)

Sau khi lưu file và chạy lại dự án:

1. Nhấn **F5** chạy Backend.
2. Sử dụng Swagger để test lại các API như ở Tiết 4.
3. Nếu dữ liệu vẫn trả về bình thường và không có lỗi hệ thống, nghĩa là cấu hình CORS đã sẵn sàng để phục vụ cho buổi học kết nối ReactJS tiếp theo.

Ghi chú cho giáo trình: "Đây là bước cuối cùng của phần Backend. Sau khi hoàn thành Tiết 5, các em đã xây dựng xong một hệ thống Web API chuẩn chỉnh, sẵn sàng cung cấp dữ liệu cho bất kỳ ứng dụng Frontend nào trên thế giới!"

Ghi chú 1: GIẢI THÍCH CÁC TRẠNG THÁI HTTP (STATUS CODES)

Mục tiêu: Giúp sinh viên hiểu cách Server "trả lời" cho Client. Trong lập trình API, Server không chỉ trả về dữ liệu mà còn gửi kèm một con số (mã trạng thái) để báo cho Frontend biết việc lấy dữ liệu đó thành công hay thất bại.

1. TẠI SAO PHẢI CÓ HTTP STATUS CODE?

Giải thích cho sinh viên:

Hãy tưởng tượng việc gọi API giống như việc bạn gửi tin nhắn cho một cửa hàng:

- Nếu họ trả lời: "Đã nhận đơn", đó là **Thành công**.
- Nếu họ trả lời: "Món này hết rồi", đó là **Lỗi phía người mua chọn sai**.
- Nếu họ không trả lời vì máy điện thoại bị hỏng, đó là **Lỗi phía cửa hàng**.

Trong lập trình, Frontend (ReactJS) dựa vào các con số này để hiển thị thông báo phù hợp cho người dùng (ví dụ: hiện dấu tích xanh khi lưu thành công, hoặc hiện thông báo "Vui lòng kiểm tra lại dữ liệu").

2. CÁC NHÓM MÃ TRẠNG THÁI PHỔ BIẾN

Giảng viên hướng dẫn sinh viên nhớ theo "đầu số" để dễ phân loại:

Nhóm 2xx: Thành công (Success)

- **200 OK:** Mọi thứ đều ổn. Server đã tìm thấy dữ liệu và gửi về cho bạn. (Dùng cho lệnh lấy danh sách, chi tiết bài viết).
- **201 Created:** Yêu cầu đã thành công và một dữ liệu mới đã được tạo ra. (Thường dùng sau khi gọi lệnh Thêm mới bài viết thành công).

Nhóm 4xx: Lỗi phía Client (Client Error)

Đây là lỗi do người gửi yêu cầu làm sai (như sai đường dẫn, thiếu dữ liệu).

- **400 Bad Request:** Server không hiểu yêu cầu hoặc dữ liệu gửi lên bị sai định dạng (ví dụ: yêu cầu nhập số nhưng gửi vào chữ).
- **401 Unauthorized:** Bạn chưa đăng nhập nên không có quyền lấy dữ liệu này.
- **404 Not Found:** Đường dẫn (URL) sai hoặc ID bài viết không tồn tại trong hệ thống.

Nhóm 5xx: Lỗi phía Server (Server Error)

Đây là lỗi do code Backend của chúng ta bị "sập".

- **500 Internal Server Error:** Một lỗi không mong muốn xảy ra trên Server (ví dụ: quên kết nối Database, code bị chia cho số 0, hoặc null reference...).
-

3. CÁCH KIỂM TRA MÃ TRẠNG THÁI TRÊN TRÌNH DUYỆT

Sinh viên cần biết cách "soi" các con số này khi làm việc thực tế:

1. Nhấn **F12** trên trình duyệt (Chrome/Edge).
2. Chọn tab **Network** (Mạng).
3. Nhấn nút chạy API (hoặc F5 lại trang).

4. Nhìn vào cột **Status**. Các dòng màu đỏ thường là lỗi (4xx, 5xx), các dòng màu đen/xanh là thành công (2xx).

4. VÍ DỤ MINH HỌA TRONG CODE (CẦM TAY CHỈ VIỆC)

Giảng viên chỉ cho sinh viên thấy các mã này xuất hiện ở đâu trong code C# của các buổi học trước:

C#

```
[HttpGet("{id}")]
public IActionResult GetDetail(int id)
{
    var post = _context.Posts.Find(id);

    if (post == null)
    {
        // Trả về mã 404
        return NotFound();
    }

    // Trả về mã 200 kèm dữ liệu
    return Ok(post);
}
```

LƯU Ý KHI GIẢNG DẠY

- **Mẹo ghi nhớ:** "2xx là OK, 4xx là do Bạn (Client), 5xx là do Tôi (Server)".
- **Tầm quan trọng:** Nhấn mạnh rằng nếu Backend trả về dữ liệu nhưng lại gửi sai mã Status (ví dụ: lỗi mà vẫn gửi mã 200), thì ứng dụng Frontend sẽ bị "loạn" và không biết phải báo lỗi như thế nào cho người dùng.

Checkpoint: Yêu cầu sinh viên thử gõ một địa chỉ API sai (ví dụ: `api/postsssss`) và quan sát mã **404** hiện ra trong tab Network. Nếu sinh viên thấy và hiểu được con số đó, các em đã nắm vững kiến thức này.

Ghi chú 2: CHECKPOINT KẾT THÚC BUỔI 6 (KIỂM TRA CUỐI GIỜ)

Mục tiêu: Đây là bước quan trọng nhất để sinh viên tự đánh giá xem hệ thống Backend API đã hoạt động đúng yêu cầu hay chưa trước khi ra về. Nếu bước này không chạy, các em sẽ không thể thực hành được Buổi 7 (ReactJS).

BƯỚC 1: KIỂM TRA DANH SÁCH TOÀN BỘ BÀI VIẾT

Thao tác:

1. Tại Visual Studio, nhấn **F5** để chạy ứng dụng.
2. Mở trình duyệt Web (Chrome, Edge...), nhập địa chỉ sau vào thanh URL:
3. `https://localhost:xxxx/api/posts`
4. (Lưu ý: xxxx là số cổng Port mà Visual Studio tự cấp cho máy của các em).

Kết quả mong đợi:

- Trình duyệt hiển thị một chuỗi văn bản dạng JSON bắt đầu bằng dấu ngoặc vuông [và kết thúc bằng].
 - Bên trong có các thông tin như: `id`, `title`, `imageUrl`, `categoryName`.
-

BƯỚC 2: KIỂM TRA CHI TIẾT MỘT BÀI VIẾT**Thao tác:**

1. Nhìn vào danh sách JSON vừa lấy được ở Bước 1, chọn một `id` bất kỳ (ví dụ: số 1 hoặc số 5).
2. Sửa địa chỉ trên thanh URL thành:
3. `https://localhost:xxxx/api/posts/1` (Thay 1 bằng ID bạn đã chọn).

Kết quả mong đợi:

- Dữ liệu trả về chỉ gồm **duy nhất một đối tượng** nằm trong dấu ngoặc nhọn {}.
 - Phải xuất hiện thêm trường `content` (Nội dung chi tiết bài viết) mà ở Bước 1 không có.
-

BƯỚC 3: KIỂM TRA TRƯỜNG HỢP LỖI (404 NOT FOUND)**Thao tác:**

1. Nhập một ID bài viết chắc chắn không tồn tại vào URL, ví dụ:
2. `https://localhost:xxxx/api/posts/999999`

Kết quả mong đợi:

- Trình duyệt hiển thị thông báo lỗi hoặc chuỗi JSON: `{"message": "Không tìm thấy bài viết này trong hệ thống"}`.
 - Nhấn **F12**, chọn tab **Network**, cột **Status** phải hiện màu đỏ với mã **404**.
-

 HƯỚNG DẪN XỬ LÝ LỖI NHANH (TROUBLESHOOTING)

Giảng viên yêu cầu sinh viên tự kiểm tra nếu gặp các trường hợp sau:

1. **Lỗi trang trắng / Không tìm thấy trang:** Kiểm tra xem đã gõ đúng chữ `api/posts` chưa (thiếu chữ `api` là lỗi phổ biến nhất).

2. **Lỗi 500 (Internal Server Error):** Thường do Database chưa có dữ liệu hoặc chuỗi kết nối (Connection String) bị sai. Hãy mở SQL Server ra kiểm tra lại bảng **Posts**.
3. **Dữ liệu hiện ra bị dính chùm, khó đọc:** Hướng dẫn sinh viên cài thêm tiện ích mở rộng (Extension) tên là **JSON Viewer** trên Chrome để dữ liệu JSON được xuống dòng và tô màu đẹp mắt, dễ quan sát.

YÊU CẦU HOÀN THÀNH:

Sinh viên chỉ được coi là hoàn thành buổi học khi **chụp màn hình** được kết quả JSON của cả 2 đường dẫn (Danh sách và Chi tiết) và dán vào báo cáo thực hành.

Tóm lại: "Dữ liệu JSON các em thấy hôm nay chính là 'nguyên liệu thô'. Buổi sau, chúng ta sẽ dùng ReactJS để biến những dòng chữ này thành một giao diện báo chí chuyên nghiệp."

===

1. **Table *CategoriesProduct* (Danh mục sản phẩm):** liệt kê danh mục các loại sản phẩm thuộc về.
Trường dữ liệu: **Id, Name, Description**
2. **Table *Products* (Sản phẩm):**
 - *Trường dữ liệu:* **Id, Name, Description, Price, StockQuantity** (Số lượng tồn), **ImageUrl, CategoryProducId** (Khóa ngoại).
3. **Table *Customers* (Khách hàng):** Lưu thông tin người mua (khác với bảng **Users** của nhân viên quản trị).
 - *Trường dữ liệu:* **Id, FullName, Email, Phone, Address, Password**.
4. **Table *Orders* (Đơn hàng):** Lưu thông tin tổng quát của một lần mua.
 - *Trường dữ liệu:* **Id, OrderDate, CustomerId** (Khóa ngoại), **Status** (0: Chờ duyệt, 1: Đang giao, 2: Đã xong), **Notes**.
5. **Table *OrderDetails* (Chi tiết đơn hàng):** Một đơn hàng có nhiều sản phẩm.
 - *Trường dữ liệu:* **Id, OrderId** (Khóa ngoại), **ProductId** (Khóa ngoại), **Quantity, UnitPrice** (Giá tại thời điểm mua).